

Capitolul 4: Proiectarea și implementarea hardware

4.1 Arhitectura generală a sistemului

4.1.1 Descrierea arhitecturii generale

Arhitectura generală a sistemului propus în cadrul acestei lucrări integrează mai multe module hardware, concepute să colaboreze pentru atingerea obiectivului principal: monitorizarea automată și inteligentă a meselor într-un restaurant. Structura sistemului reflectă o abordare modulară, în care sarcinile de procesare și control sunt împărțite între diferite componente specializate.

În centrul arhitecturii se află patru blocuri funcționale principale: un calculator personal (PC) care acționează ca platformă de procesare software, o cameră video montată pe un sistem motorizat cu două axe, o placă de circuit imprimat (PCB) personalizată ce integrează microcontrolerul ESP32-S3-MINI și o sursă externă de alimentare de 12V/2A. Fiecare dintre aceste blocuri are un rol clar definit în cadrul sistemului, fiind interconectate atât la nivel fizic, cât și la nivel logic.

Camera video, capabilă să furnizeze imagini de înaltă calitate, este montată pe un mecanism motorizat cu două grade de libertate, acționat prin intermediul a două servomotoare. Controlul poziției camerei este realizat prin comenzi transmise de PC către ESP32, care generează semnalele necesare pentru mișcarea pe axele O_x și O_y . Platforma software de pe PC rulează algoritmul de detecție bazat pe rețeaua YOLO, având rolul de a analiza fluxul video și de a determina statusul meselor monitorizate.

Această arhitectură distribuie clar responsabilitățile între partea software și partea hardware, favorizând modularitatea și facilitând extinderea ulterioară a sistemului. Comunicarea dintre module se realizează printr-o conexiune USB-C, iar sursa de alimentare asigură tensiunile necesare funcționării atât a microcontrolerului, cât și a servomotoarelor.

4.1.2 Schema logică funcțională

Schema logică funcțională a sistemului este ilustrată în Figura 4.1 și prezintă modul în care principalele componente hardware interacționează pentru a susține funcționarea generală a proiectului. Această schemă evidențiază conexiunile esențiale între modulele hardware, precum și traseele de comunicație și alimentare care asigură integrarea funcțională a întregului ansamblu.

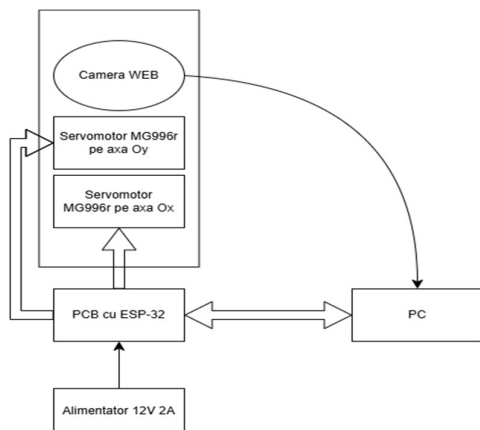


Figura 4.1 – Schema bloc

Conform Figura 4.1, sistemul este organizat în patru blocuri majore: camera Intel RealSense D435i montată pe un sistem motorizat cu două servomotoare MG996R (controlate pe axele Ox și Oy), o placă PCB echipată cu microcontroler ESP32, un calculator personal (PC) și o sursă de alimentare de 12V/2A. Fiecare dintre aceste componente este reprezentată schematic și conectată logic în fluxul de informații și energie.

Fluxul de date începe de la camera Intel RealSense D435i, care captează imagini din mediul monitorizat și le transmite direct către PC pentru procesare. PC-ul analizează fluxul video prin intermediul unui algoritm de detecție bazat pe YOLO și decide, în funcție de rezultatele procesării, dacă este necesară re poziționarea camerei. În cazul în care se impune ajustarea unghiului de vizualizare, PC-ul transmite comenzi către placa PCB cu ESP32 prin interfața USB-C, reprezentată în schema logică printr-o săgeată bidirecțională între PC și PCB.

Placa PCB primește aceste comenzi în format serial (convertite intern de la USB la UART) și, prin intermediul bibliotecii ESP32Servo, generează semnalele PWM necesare pentru controlul celor două servomotoare MG996R. Acestea sunt responsabile pentru orientarea camerei pe axele Ox și Oy, conform indicațiilor din schema funcțională. Traseul semnalelor de control este realizat prin conexiuni directe între pini specifici ai microcontrolerului și intrările de semnal ale celor două servomotoare, așa cum este reprezentat în schema logică.

De asemenea, schema logică prezintă circuitul de alimentare: sursa de 12V/2A furnizează tensiunea necesară întregului sistem, iar aceasta este distribuită către placa PCB, care reglează local tensiunile necesare pentru alimentarea servomotoarelor și a microcontrolerului. Legătura dintre sursa de alimentare și PCB este evidențiată printr-o conexiune directă în schema ilustrată.

În ansamblu, schema logică funcțională prezentată în Figura 4.1 sintetizează relațiile dintre componente, traseele de semnale și alimentare, precum și fluxul operațional al informațiilor, constituind o bază esențială pentru implementarea fizică și funcțională a sistemului.

4.1.3 Relația dintre componentele hardware

Relația dintre componentele hardware este definită atât de legăturile fizice dintre ele, cât și de rolurile complementare pe care le îndeplinesc în cadrul sistemului. Fiecare componentă este esențială pentru funcționarea coerentă a ansamblului, contribuind cu o funcție specifică într-un lanț operațional integrat.

Camera video are rolul de a capta imagini din mediul monitorizat, fiind montată pe un suport motorizat care permite orientarea în plan orizontal (Ox) și vertical (Oy). Mișcarea este realizată cu ajutorul a două servomotoare de tip MG996R, fiecare fiind alocat controlului unei singure axe. Aceste servomotoare sunt conectate direct la placa PCB prevăzută cu un microcontroler ESP32-S3-MINI, care generează semnalele PWM necesare pentru stabilirea precisă a unghiurilor de rotație, prin intermediul bibliotecii software ESP32Servo.

Placa PCB joacă rolul unui centru de control hardware, interfațând camera motorizată cu platforma de procesare software. Aceasta integrează microcontrolerul ESP32-S3-MINI, circuitele de alimentare și protecție, precum și interfețele de conectare necesare. Prin portul USB-C, placa PCB realizează comunicarea bidirecțională cu PC-ul, permițând recepția comenzilor în format serial și generarea corespunzătoare a semnalelor PWM necesare pentru controlul servo-motoarelor, prin intermediul codului firmware. Totodată, portul USB oferă un punct de acces facil pentru programare și depanare.

Sursa de alimentare externă de 12V/2A este conectată la PCB, asigurând energia necesară atât pentru funcționarea servomotoarelor, cât și pentru alimentarea microcontrolerului. Conversia locală a tensiunilor pe PCB permite distribuirea controlată a energiei către fiecare subsistem, evitând supratensiunile sau instabilitatea.

Prin această relație strânsă și coordonată între componente, sistemul asigură o funcționare sincronizată, în care fiecare element contribuie la realizarea obiectivului global de monitorizare automată și dinamică a meselor.

4.2 Componente hardware

4.2.1 Microcontrolerul ESP32-S3-MINI

ESP32-S3-MINI reprezintă un modul de ultimă generație dezvoltat de compania Espressif Systems, bazat pe microcontrolerul ESP32-S3. Este conceput pentru aplicații care impun un echilibru optim între putere de procesare, conectivitate wireless și dimensiuni reduse. Acest modul integrează atât funcționalități Wi-Fi, cât și Bluetooth Low Energy (LE), oferind o soluție completă pentru sisteme embedded, dispozitive IoT și aplicații portabile.



ESP32-S3-MINI-1

Figura 4.2 - ESP32-S3-MINI, sursa: [39]

În Figura 4.2 este prezentată o vedere tridimensională a modului ESP32-S3-MINI, ce evidențiază designul său compact, precum și distribuția componentelor pe suprafața plăcii. Designul urmărește maximizarea densității funcționale într-un spațiu redus, facilitând integrarea modului în plăci de circuit imprimate personalizate, unde spațiul disponibil este adesea limitat.

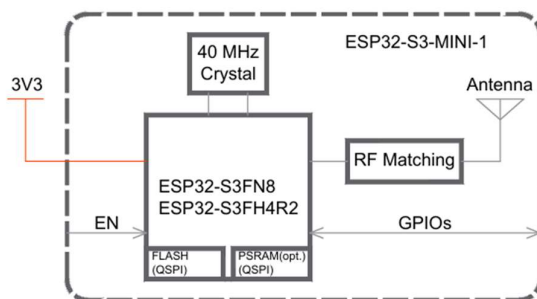


Figura 4.3 - diagramă bloc ESP32-S3-MINI, sursa: [39]

Arhitectura internă a modului este ilustrată în Figura 4.3, printr-o diagramă bloc ce pune în evidență conexiunile dintre principalele componente hardware: procesorul dual-core Xtenso® LX7, memoria flash externă, PSRAM-ul opțional, perifericele digitale și analogice, precum și interfețele de comunicație. Această arhitectură modulară permite procesarea paralelă a sarcinilor, gestionarea eficientă a datelor și suportul pentru mai multe protocoale de comunicație simultan.

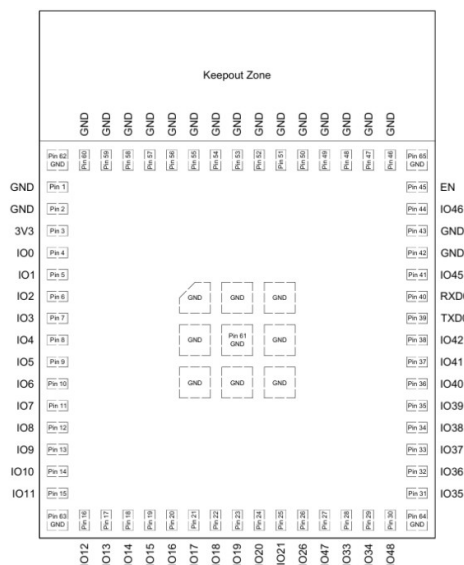


Figura 4.4 - pinout-ului modulului ESP32-S3-MINI, sursa: [39]

Pentru o mai bună înțelegere a resurselor disponibile, Figura 4.4 prezintă schema pinout-ului modulului ESP32-S3-MINI. Această schemă indică poziționarea și funcționalitatea fiecărui pin de intrare/ieșire, oferind informații esențiale pentru conectarea modulelor periferice, a senzorilor și a altor componente externe.

Specificarea completă a caracteristicilor tehnice ale modulului ESP32-S3-MINI este detaliată în Tabelul 4.1, conform documentației oficiale furnizate de Espressif Systems [39].

Caracteristică	Specificație
Procesor	Xtensa® 32-bit LX7 dual-core, până la 240 MHz
Memorie internă	512 KB SRAM, 384 KB ROM, 16 KB SRAM în RTC
Memorie externă	4 MB flash și 2 MB PSRAM sau 8 MB flash fără PSRAM
Conectivitate wireless	Wi-Fi 802.11b/g/n, Bluetooth 5 (LE)
Interfețe periferice	GPIO, ADC, DAC, I2C, SPI, UART, USB OTG
Tensiune de alimentare	3.0 – 3.6 V
Dimensiuni fizice	15.4 mm × 20.5 mm × 2.4 mm (cu antenă PCB integrată)
Temperatura de operare	-40°C până la +85°C

Tabelul 4.1 – Specificații tehnice ale modulului ESP32-S3-MINI [39]

Aceste specificații confirmă capacitatea modulului ESP32-S3-MINI de a susține aplicații complexe, care implică procesare locală rapidă, conectivitate multiplă și interacțiune cu periferice variate. Dimensiunile sale reduse, combinate cu suportul pentru interfețe multiple (GPIO-uri, ADC, DAC, I2C, SPI, UART, USB OTG), fac din acest modul o soluție extrem de flexibilă și adaptabilă.

În plus, integrarea nativă a funcționalităților Wi-Fi și Bluetooth LE contribuie la extinderea potențialului de conectivitate al sistemelor în care este utilizat, eliminând necesitatea modulelor externe suplimentare. Această abordare reduce atât costurile de implementare, cât și complexitatea generală a designului hardware.

Documentația oficială evidențiază și eficiența energetică a modulului, precum și suportul extins din partea comunității de dezvoltatori și a resurselor open-source disponibile online, aspecte esențiale pentru accelerarea procesului de dezvoltare și prototipare [39].

4.2.2 Camera Intel RealSense D435i



Figura 4.5 - camera Intel RealSense D435i, sursa: [40]

În cadrul sistemului de monitorizare, un rol esențial este deținut de camera Intel RealSense D435i, ilustrată în Figura 4.5. Aceasta face parte din seria D400 și oferă funcționalități avansate de captare a adâncimii și a imaginilor RGB. Camera D435i combină un senzor stereo activ, un proiector infraroșu și un IMU integrat pentru a furniza simultan imagini 3D și date de mișcare [39].

S-a utilizat exclusiv fluxul camerei RGB integrate, fără a exploata momentan capabilitățile de adâncime sau datele IMU. Alegerea s-a bazat pe necesitatea obținerii unor imagini clare în timp real, esențiale pentru procesarea prin YOLO. Camera D435i poate furniza imagini la 1920×1080 pixeli, cu până la 30 cadre pe secundă, permițând capturi de înaltă calitate într-un cadru dinamic [41].

În perspectivă, se dorește extinderea funcționalității prin integrarea datelor de adâncime în algoritmul de detecție a statusului meselor, pentru a îmbunătăți precizia în situații cu suprapuneri sau obstacole. Camera D435i permite măsurători de adâncime pe distanțe între 0,1 m și 10 m, oferind flexibilitate sporită în analiza tridimensională a scenelor [42]. Mai jos este prezentat Tabelul 4.2:

Specificație	Valoare
Tehnologie adâncime	Stereo activ cu proiector IR
Câmp vizual (FOV)	$86^\circ \times 57^\circ \times 95^\circ$
Rezoluție maximă RGB	1920×1080 pixeli
Cadre pe secundă (FPS)	Până la 90 FPS (la rezoluții mici)
Interval măsurare adâncime	0,1 m – 10 m
IMU integrat	Da
Conectivitate	USB 3.1 Type-C
Dimensiuni	$90 \text{ mm} \times 25 \text{ mm} \times 25 \text{ mm}$
Greutate	72 grame
Temperatura operare	$0^\circ\text{C} - 35^\circ\text{C}$

Tabel 4.2 – Date tehnice ale camerei Intel RealSense D435i

Integrarea camerei, prezentată în Figura 4.5, a oferit o soluție optimă pentru captarea imaginilor necesare detecției automate a meselor. Utilizarea exclusivă a fluxului RGB a permis o

integrare rapidă în aplicația AdminGUI, iar pe viitor, utilizarea simultană a canalelor RGB și de adâncime ar putea contribui la creșterea acurateții sistemului.

D435i oferă suport software extins pentru OpenCV și RealSense SDK, facilitând integrarea datelor vizuale și de adâncime în aplicații avansate de viziune computerizată [43]. Prin aceste capabilități, camera rămâne un element cheie în perspectiva extinderii funcționalităților sistemului.

4.2.3 ServoMotoarele MG996R

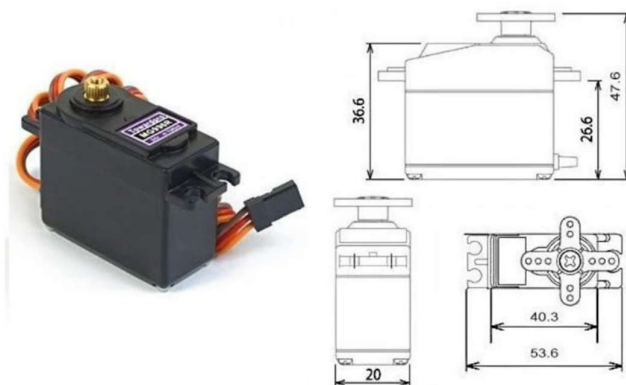


Figura 4.6 – servomotorul MG996R, sursa: [44]

Un alt element esențial în componența sistemului hardware al proiectului este servomotorul MG996R, ilustrat în Figura 4.6. Acesta a fost utilizat pentru acționarea mecanismului de orientare a camerei Intel RealSense D435i, permițând astfel ajustarea unghiului de vizualizare al zonei monitorizate în funcție de necesități.

MG996R face parte din categoria servomotoarelor digitale de cuplu mare, fiind o soluție accesibilă și fiabilă în aplicații de robotică și automatizare. Potrivit documentației oficiale, „MG996R este un servomotor digital de înaltă performanță, cu un cuplu maxim de 11 kg·cm la 6 V și o viteză de rotație de 0,17 secunde per 60°” [44]. Această caracteristică îl face potrivit pentru aplicații ce necesită control precis al poziției și o forță mecanică semnificativă, cum este cazul manipulării unei camere montate pe un braț sau suport.

Motorul este echipat cu angrenaje metalice, ceea ce îi conferă o rezistență sporită la uzură și o durată de viață mai mare în comparație cu motoarele similare cu angrenaje din plastic. „Construcția robustă cu angrenaje metalice asigură fiabilitatea motorului în condiții de sarcină mecanică ridicată, reducând riscul de deteriorare a transmisiei interne” [45].

Din punct de vedere al specificațiilor electrice, MG996R funcționează la o tensiune de alimentare standard de 4,8–7,2 V, având un consum de curent în gol de aproximativ 500 mA și un curent maxim ce poate atinge 2,5 A sub sarcină maximă [46]. Aceste valori impun utilizarea unui circuit de alimentare capabil să furnizeze curenți mari pe perioade scurte, pentru a evita pierderi de cuplu sau resetări accidentale.

Mai jos este prezentat Tabelul 4.3, realizat pe baza informațiilor furnizate în fișa tehnică oficială [44]:

Specificație	Valoare
Cuplu maxim	11 kg·cm la 6 V
Viteză rotație	0,17 s/60° la 6 V
Tensiune alimentare	4,8 – 7,2 V
Tip control	PWM
Unghi maxim rotație	120° – 180°
Material angrenaje	Metal
Dimensiuni	40,7 × 19,7 × 42,9 mm
Greutate	55 grame

Tabel 4.3 – Date tehnice ale servomotorului MG996R, sursa: [44]

Integrarea servomotorului MG996R în sistemul hardware a permis obținerea unui mecanism de orientare fiabil, capabil să susțină greutatea camerei Intel RealSense D435i și să asigure o mișcare precisă în plan orizontal. „MG996R suportă un unghi de rotație între 120° și 180°, în funcție de impulsurile PWM aplicate, oferind flexibilitate în controlul poziției” [47].

Prin caracteristicile sale tehnice și prin raportul cost-performanță avantajos, servomotorul MG996R s-a dovedit o alegere potrivită pentru sistemul dezvoltat, asigurând un echilibru între putere, precizie și fiabilitate.

4.2.4. Alimentatorul universal 12V 2A



Figura 4.7 - Alimentatorul universal 12V 2A, sursa:[48]

În cadrul sistemului hardware dezvoltat, alimentatorul universal 12V 2A, prezentat în Figura 4.7, joacă un rol esențial în asigurarea tensiunii și curentului necesar pentru funcționarea ansamblului electronic, în special a servomotoarelor MG996R și a plăcii de control ESP32-S3-MINI.

Acest alimentator furnizează o tensiune stabilizată de 12V continuu, având o capacitate de curent de până la 2A, suficientă pentru a susține simultan consumatorii principali ai sistemului. Potrivit documentației oficiale, „alimentatorul universal 12V 2A este proiectat pentru a oferi o sursă de energie fiabilă și stabilă pentru echipamente electronice de mică și medie putere” [48].

Un aspect important al utilizării acestui alimentator este legat de protecțiile integrate. „Dispozitivul este echipat cu protecție la supratensiune, supracurent și scurtcircuit, prevenind deteriorarea componentelor alimentate în cazul unor defecțiuni electrice” [48]. Aceste caracteristici

de siguranță sunt esențiale mai ales în aplicațiile ce implică motoare, unde fluctuațiile de curent sunt frecvente.

Alimentatorul beneficiază de un conector tip jack de 5,5 mm, compatibil cu majoritatea modulelor electronice standard. În cadrul proiectului, s-a utilizat un adaptor jack la terminal cu șuruburi, pentru a conecta direct cablurile de alimentare ale motoarelor și ale plăcii ESP32-S3-MINI. „Conectorul standardizat de 5,5 mm permite integrarea facilă a alimentatorului în diverse circuite fără modificări mecanice suplimentare” [48].

Din punct de vedere constructiv, carcasa este realizată din plastic ignifug, având dimensiuni reduse și greutate mică, aspecte ce contribuie la ușurința instalării în spații restrânse. Potrivit specificațiilor, „alimentatorul prezintă o eficiență energetică ridicată, de aproximativ 85%, reducând pierderile de căldură în regim continuu de funcționare” [48].

Mai jos este prezentat Tabelul 4.4, pe baza informațiilor din documentația oficială [48]:

Specificație	Valoare
Tensiune de ieșire	12V DC
Curent maxim	2A
Putere maximă	24W
Protecții integrate	Supratensiune, supracurent, scurtcircuit
Eficiență energetică	85%
Conector ieșire	Jack 5,5 mm
Dimensiuni	80 × 50 × 30 mm
Greutate	120 grame

Tabel 4.4 – Date tehnice ale alimentatorului universal 12V 2A, sursa: [48]

Integrarea acestui alimentator în sistemul hardware a permis alimentarea sigură și stabilă a componentelor, contribuind la menținerea unei funcționări optime a servomotoarelor și a plăcii de control. Alegerea sa s-a bazat pe raportul cost-eficiență și pe compatibilitatea cu cerințele de curent și tensiune ale echipamentelor utilizate.

4.2.5 Stabilizatorul de tensiune LM1085ISX-5.0/NOPB



Figura 4.8 - LM1085ISX-5.0/NOPB, sursa: [49]

Pentru a asigura o alimentare stabilă și fiabilă a componentelor electronice sensibile din cadrul sistemului, s-a utilizat stabilizatorul de tensiune LM1085ISX-5.0/NOPB, prezentat în Figura 4.8. Acest regulator liniar de tensiune, fabricat de Texas Instruments, oferă o ieșire fixă de 5V și un curent maxim de 3A, fiind ideal pentru aplicații care necesită o sursă de tensiune constantă și curent ridicat.

Conform fișei tehnice oficiale, „LM1085 este un regulator cu un dropout maxim de 1,5 V la un curent de sarcină de 3 A” [50]. Această caracteristică îl face potrivit pentru aplicații unde diferența dintre tensiunea de intrare și cea de ieșire trebuie să fie minimă, reducând astfel disiparea de putere și îmbunătățind eficiența energetică.

Dispozitivul este echipat cu protecții interne împotriva supracurentului și supraîncălzirii, asigurând o funcționare sigură în condiții variate de sarcină. „Circuitul LM1085 include o referință bandgap ajustată cu zener, limitare de curent și oprire termică” [50]. Aceste funcții de protecție sunt esențiale pentru prevenirea deteriorării componentelor în cazul unor condiții de operare nefavorabile.

Mai jos este prezentat Tabelul 4.5, conform documentației oficiale:

Specificație	Valoare
Tensiune de ieșire	5V (fixă)
Curent maxim de ieșire	3A
Tensiune maximă de intrare	25V
Dropout maxim	1,5V la 3A
Protecții integrate	Supracurent, supraîncălzire
Tip pachet	TO-263 (DDPAK-3)
Interval temperatură operare	-40°C până la +125°C
PSRR (120Hz)	68 dB

Tabel 4.5 – Date tehnice ale stabilizatorului LM1085ISX-5.0/NOPB, sursa: [50]

Integrarea stabilizatorului LM1085ISX-5.0/NOPB în sistemul hardware a permis obținerea unei alimentări stabile de 5V pentru componentele critice, contribuind la fiabilitatea și performanța generală a proiectului.

4.2.6 Stabilizatorul de tensiune TL1963A-33DCYR

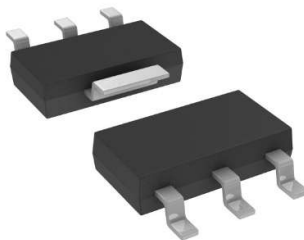


Figura 4.9 - TL1963A-33DCYR, sursa: [51]

Pentru a asigura o alimentare stabilă și fiabilă a componentelor electronice sensibile din cadrul sistemului, s-a utilizat stabilizatorul de tensiune TL1963A-33DCYR, prezentat în Figura 4.9. Acest

regulator liniar de tensiune, fabricat de Texas Instruments, oferă o ieșire fixă de 3,3V și un curent maxim de 1,5A, fiind ideal pentru aplicații care necesită o sursă de tensiune constantă și curent moderat.

Conform fișei tehnice oficiale, „TL1963A-33 este un regulator cu un dropout maxim de 340 mV la un curent de sarcină de 1,5 A” [52]. Această caracteristică îl face potrivit pentru aplicații unde diferența dintre tensiunea de intrare și cea de ieșire trebuie să fie minimă, reducând astfel disiparea de putere și îmbunătățind eficiența energetică [52].

Dispozitivul este echipat cu protecții interne împotriva supracurentului, supraîncălzirii, tensiunii inverse și curentului invers, asigurând o funcționare sigură în condiții variate de sarcină. „Circuitul TL1963A-33 include protecție la supracurent, limitare termică, protecție la tensiune inversă și protecție la curent invers” [52]. Aceste funcții de protecție sunt esențiale pentru prevenirea deteriorării componentelor în cazul unor condiții de operare nefavorabile [52].

Mai jos este prezentat Tabelul 4.6, conform documentației oficiale [52]:

Specificație	Valoare
Tensiune de ieșire	3,3V (fixă)
Curent maxim de ieșire	1,5A
Tensiune maximă de intrare	20V
Dropout maxim	340 mV la 1,5A
Protecții integrate	Supracurent, supraîncălzire, tensiune inversă, curent invers
Tip pachet	SOT-223-4
Interval temperatură operare	-40°C până la +125°C
PSRR (120Hz)	63 dB

Tabel 4.6 – Date tehnice ale stabilizatorului TL1963A-33DCYR, sursa: [52]

Integrarea stabilizatorului TL1963A-33DCYR în sistemul hardware a permis obținerea unei alimentări stabile de 3,3V pentru componentele critice, contribuind la fiabilitatea și performanța generală a proiectului.

4.2.7 Convertor USB–UART FT231XQ-R

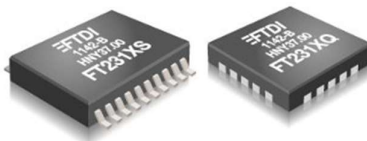


Figura 4.10 - USB–UART FT231XQ-R

Pentru a facilita comunicarea între microcontrolerul ESP32-S3-MINI și un computer personal prin interfață USB, în cadrul sistemului hardware a fost integrat circuitul FT231XQ-R, prezentat în Figura 4.10. Acest circuit integrat, fabricat de FTDI, oferă o soluție compactă pentru conversia

semnalelor USB 2.0 Full Speed în semnale UART, eliminând necesitatea unei programări suplimentare a firmware-ului USB.

Conform fișei tehnice oficiale, „FT231XQ-R este un convertor USB 2.0 Full Speed către interfață UART cu handshake complet, capabil să opereze până la 3 Mbps” [53]. Dispozitivul include o memorie MTP integrată și un circuit de ceas intern, eliminând necesitatea unui cristal extern și reducând astfel numărul de componente externe necesare. De asemenea, consumul de curent este redus, având un consum tipic de 8 mA în regim activ și 125 μ A în modul suspendat.

Mai jos este prezentat Tabelul 4.7, conform documentației oficiale [53]:

Specificație	Valoare
Interfață USB	USB 2.0 Full Speed
Viteză de transfer UART	Până la 3 Mbps
Tensiune de alimentare	3,3V – 5,5V
Curent de consum tipic	8 mA
Curent în modul suspendat	125 μ A
Pachet	QFN-20 (4x4 mm)
Temperatura de operare	-40°C până la +85°C
Funcții suplimentare	EEPROM integrat, ceas intern

Tabel 4.7 – Date tehnice ale convertorului FT231XQ-R, sursa: [53]

Integrarea convertorului FT231XQ-R în sistemul hardware a permis o comunicare eficientă și fiabilă între microcontroler și computer, contribuind la ușurința în programare și diagnosticare a sistemului.

4.2.8 Conector USB-C Molex 217179-0001



Figura 4.11 - USB-C Molex 217179-0001, sursa: [54]

Pentru a asigura o interfață de alimentare și comunicație modernă și robustă în cadrul sistemului hardware, a fost integrat conectorul Molex 217179-0001, prezentat în Figura 4.11. Acest conector USB 2.0 de tip C, cu montare la unghi drept și fixare prin tehnologie SMD, oferă o soluție compactă și fiabilă pentru aplicații electronice moderne.

Conform fișei tehnice oficiale, „conectorul 217179-0001 este un receptacol USB 2.0 de tip C, cu 24 de poziții (16 active și 8 neconectate), montare la unghi drept și fixare prin tehnologie SMD”

[54]. Acesta suportă un curent maxim de 5A și o tensiune de până la 30V AC/DC, fiind ideal pentru aplicații care necesită o alimentare eficientă și transfer de date fiabil.

Mai jos este prezentat Tabelul 4.8, conform documentației oficiale [54]:

Specificație	Valoare
Tip conector	USB 2.0 Type-C
Număr de contacte	24 (16 active + 8 neconectate)
Montare	SMD, unghi drept
Curent maxim	5A
Tensiune maximă	30V AC/DC
Durabilitate	10.000 cicluri de cuplare
Temperatura de operare	-40°C până la +85°C
Material contacte	Aliaj de cupru, placat cu aur
Material carcasă	Oțel inoxidabil

Tabel 4.8 – Date tehnice ale conectorului Molex 217179-0001, sursa: [54]

Integrarea conectorului Molex 217179-0001 în sistemul hardware a permis o conectivitate fiabilă și durabilă, asigurând compatibilitatea cu standardele moderne USB și contribuind la eficiența generală a proiectului.

4.3 Schema electrică și implementarea hardware detaliată

4.3.1 Introducere generală și rolul plăcii PCB

Realizarea unei plăci PCB personalizate a constituit un element fundamental în arhitectura hardware a sistemului de monitorizare automată dezvoltat în cadrul proiectului. Această placă nu servește doar ca suport fizic pentru componentele electronice, ci și ca un hub central de alimentare, interfațare și conectivitate pentru întreg sistemul bazat pe microcontrolerul ESP32-S3-MINI-1-N8. În acest subcapitol se descrie în detaliu rolul funcțional al plăcii și structura sa electrică, respectând schema reală implementată.

Placa a fost proiectată utilizând software-ul Altium Designer și este realizată pe patru straturi. Această configurație multi-layer a permis separarea eficientă a traseelor electrice și a planurilor de alimentare și masă. Structura straturilor este organizată astfel:

- Layer 1 (roșu): trasee pentru liniile de +12V, +5V, semnale de comunicație USB și conexiuni diferențiale între portul USB-C și convertorul FT231XQ-R.
- Layer 2 (galben): trasee dedicate semnalelor de +3,3V și rutărilor suplimentare neacoperite pe primul strat.
- Layer 3 (mov): plan complet de masă, utilizat pentru întreg PCB-ul, asigurând stabilitatea electrică și reducerea interferențelor.
- Layer 4 (albastru): trasee pentru semnalele GPIO de la microcontroler către conectorii de extensie, alături de alte semnale auxiliare.

Un aspect important al designului este integrarea zonelor de cupru suplimentare pe straturile de semnal, utilizate ca heatsink pentru disiparea eficientă a căldurii generate de stabilizatoarele de tensiune.

Microcontrolerul ESP32 este alimentat din două surse alternative: fie dintr-o intrare externă de +12V, redusă succesiv la +5V prin stabilizatorul LM1085ISX-5.0/NOPB și apoi la +3,3V prin stabilizatorul TL1963A-33DCYR, fie prin portul USB-C, în cazul activării jumperului 5VBUS→5V. Activarea jumperului conectează linia VBUS de la USB la intrarea stabilizatorului de 5V, permițând alimentarea completă a plăcii prin USB. În lipsa jumperului, alimentarea prin USB este utilizată exclusiv pentru comunicație și programare.

Blocul de comunicație USB–UART este realizat cu integratul FT231XQ-R, conectat la portul USB-C. Semnalele diferențiale USB_D+ și USB_D- sunt filtrate și protejate de diodele AQ3045-01ETG și trase prin rezistențe de 27Ω (R5, R6), asigurând potrivirea de impedanță și reducerea zgomotului. Circuitul generează semnalele TXD_FTDI și RXD_FTDI, transmise către ESP32 prin rezistențe 0Ω (R14, R15), utilizate ca legături directe sau jumperi de configurare.

Pentru controlul inițializării automate și a resetării, semnalele RTS_FTDI și DTR_FTDI sunt rutate prin tranzistori SS8050 (Q2, Q3) și rezistențe 10kΩ (R1, R4), conform schemei, permițând activarea pinilor CHIP_PU și GPIO0 fără intervenție manuală. Opțional, placa include două butoane tactile RESET (SW1) și BOOT (SW2), conectate la aceiași pini, pentru control manual.

Pentru feedback vizual, sunt integrate două LED-uri de stare (D6 – verde și D7 – roșu), fiecare conectat la +3,3V prin rezistențe de 1kΩ (R12, R13), utile pentru diagnosticarea rapidă a funcționării plăcii.

Placa expune o serie de pini GPIO prin conectorii J3 și J4, oferind acces direct la majoritatea pinilor microcontrolerului. În plus, conectorii J5 și J6 asigură acces suplimentar la GPIO6 și GPIO7, alături de alimentare +5V și GND, destinați extinderilor hardware.

În zona de protecție, schema include un tranzistor IRF5305PBF (Q1), o diodă Zener 10V (D1) și o diodă TVS AVR-M1608C120MT6AB (D2), pentru protecția liniilor de alimentare la supratensiuni și comutare controlată.

4.3.2 Prezentarea generală a schemei electrice

Schema electrică a plăcii PCB personalizate din cadrul acestui proiect integrează o serie de blocuri funcționale interconectate, fiecare având un rol bine definit în asigurarea funcționării optime a sistemului de monitorizare automată. În această secțiune este prezentată o imagine de ansamblu asupra acestor blocuri, evidențiind structura logică și relațiile dintre componentele principale. Detalierea fiecărui bloc va fi realizată în subcapitolele următoare.

Primul bloc esențial al schemei este blocul de alimentare, responsabil pentru gestionarea și stabilizarea tensiunilor necesare operării circuitelor. Acesta integrează două stabilizatoare liniare: unul pentru reducerea tensiunii de intrare de +12V la +5V și al doilea pentru conversia de la +5V la +3,3V, tensiune utilizată de microcontroler și de restul componentelor logice. Blocul de alimentare include și componente de protecție, precum siguranțe, diode TVS și zener, precum și condensatori de filtrare pentru stabilizarea tensiunilor.

Un alt bloc important este blocul de conversie USB–UART, realizat prin integratul FT231XQ-R, care permite comunicarea și programarea microcontrolerului prin intermediul unui port USB-C.

Acest bloc asigură transformarea semnalelor USB în semnale UART compatibile cu ESP32 și integrează circuite auxiliare pentru controlul resetării și modului de boot al microcontrolerului.

În centrul schemei se află blocul de microcontroler, reprezentat de modulul ESP32-S3-MINI-1-N8, conectat la o serie extinsă de pini GPIO expuși prin conectori externi. Acest bloc este completat de circuite auxiliare de inițializare, control și protecție, inclusiv butoanele RESET și BOOT, precum și rezistențele de pull-up și pull-down necesare configurării inițiale.

Schema mai include și un bloc de semnalizare vizuală, alcătuit din două LED-uri conectate prin rezistențe de limitare a curentului, utilizate pentru indicarea stării plăcii sau a altor evenimente definite software.

Pentru interfațarea cu componente externe, placa oferă un bloc de extensie GPIO, realizat prin conectori suplimentari care expun pinii GPIO6 și GPIO7, împreună cu linii de alimentare +5V și GND, destinate extinderilor hardware viitoare sau conectării unor periferice suplimentare.

În completarea acestor blocuri, schema integrează un bloc de protecție și comutare, care include tranzistorul IRF5305PBF, o diodă zener și circuite asociate, menite să protejeze și să controleze liniile de alimentare și semnal în diverse condiții de operare.

Toate aceste blocuri sunt interconectate prin trasee plasate pe o arhitectură cu 4 straturi, fiecare strat având un rol funcțional bine definit: primul strat alocând traseele de alimentare și comunicație USB, al doilea strat semnalelor de +3,3V, al treilea strat servind ca plan complet de masă, iar al patrulea strat fiind destinat semnalelor GPIO și altor trasee suplimentare.

Astfel, schema electrică prezintă o structură modulară și clar organizată, fiecare bloc având o funcție bine delimitată, contribuind la realizarea unui sistem hardware fiabil, protejat și ușor de extins. În subcapitolele următoare vor fi prezentate detaliat fiecare dintre aceste blocuri, incluzând explicații privind componentele utilizate, traseele implicate și interacțiunile interne.

4.3.3 Blocul de alimentare principală

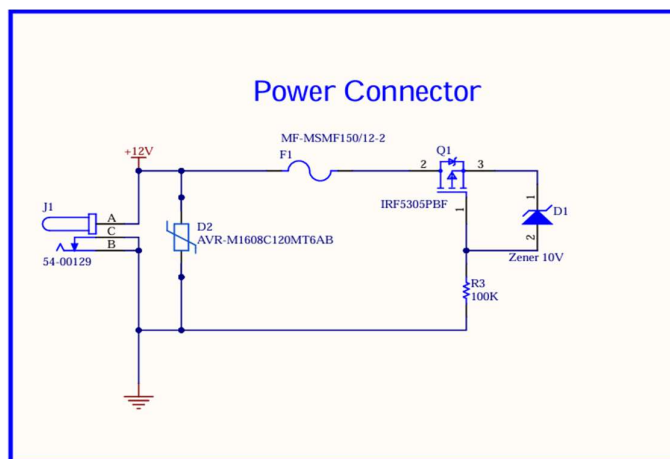


Figura 4.12 – Blocul de alimentare principală

Blocul de alimentare principală are rolul de a prelua tensiunea externă de +12V și de a asigura protecția sistemului împotriva supracurenților și supratensiunilor, după cum este ilustrat în Figura 4.12. Acesta este prezentat în Figura 4.11 și include următoarele componente esențiale: conectorul de alimentare J1, siguranța F1, dioda TVS D2, tranzistorul Q1, dioda Zener D1 și rezistența R3.

Conectorul J1 este un port de alimentare tip baril care acceptă un adaptor de +12V, tensiune ce reprezintă sursa principală de energie pentru întreaga placă. Imediat după conector, tensiunea trece prin siguranța resetabilă F1 (tip PTC), care întrerupe automat curentul în cazul unei suprasarcini și revine la starea inițială după răcire.

Pentru protecția împotriva supratensiunilor tranzitorii (ESD, vârfuri de tensiune), pe linia de +12V este conectată o diodă TVS (D2). Aceasta are rolul de a limita tensiunea prin descărcarea rapidă a energiilor dăunătoare direct către masă.

Tensiunea protejată ajunge apoi la un tranzistor MOSFET canal P (IRF5305PBF, Q1), care funcționează ca întrerupător controlat. Comanda porții tranzistorului este realizată cu ajutorul unei diodă Zener de 10V (D1) și a unei rezistențe de 100kΩ (R3). Acest circuit stabilizează tensiunea pe poartă și menține tranzistorul în regim de conducție cât timp tensiunea de intrare este sigură. Dacă tensiunea depășește pragul de funcționare al diodei zener, tranzistorul se va opri, blocând alimentarea către restul circuitului.

Prin această arhitectură, blocul de alimentare asigură:

- Protecție la supracurent (siguranță F1),
- Protecție la supratensiune tranzitorie (diodă TVS),
- Filtrare și control activ al alimentării (MOSFET + Zener).

Acest bloc pregătește tensiunea de intrare pentru a fi transmisă către stabilizatorul de 5V, oferind astfel o bază solidă și protejată pentru întregul sistem electronic al plăcii.

4.3.4 Blocul de conversie 12V la 5V

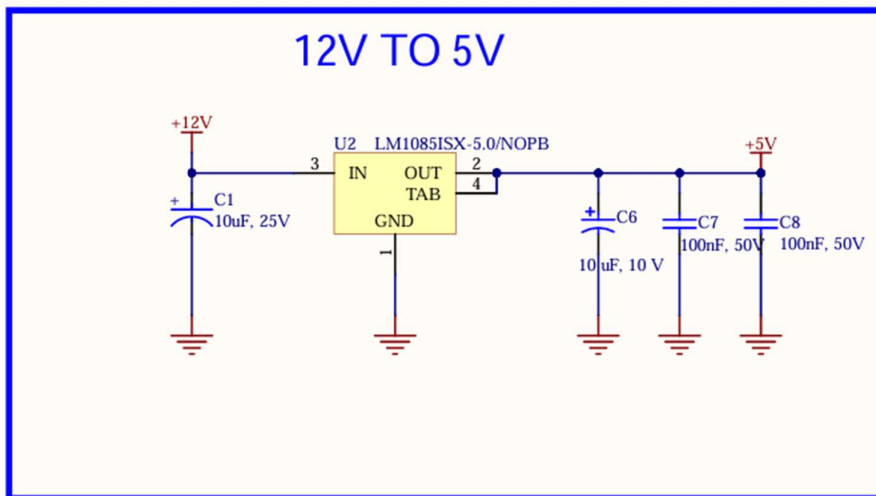


Figura 4.13 - Blocul de conversie 12V la 5V

După ce tensiunea de +12V a fost filtrată și protejată de blocul anterior, următorul pas în arhitectura electrică a plăcii îl reprezintă conversia eficientă la +5V, necesară pentru alimentarea intermediară a microcontrolerului și a altor componente auxiliare. Acest proces este realizat cu ajutorul unui regulator de tensiune liniar, LM1085ISX-5.0/NOPB, ilustrat în Figura 4.13.

Stabilizatorul U2 este un regulator liniar de joasă tensiune, capabil să furnizeze până la 3A la ieșire, fiind astfel ideal pentru aplicații care necesită o alimentare curată și stabilă. Acesta este conectat cu trei terminale: IN (intrare), OUT (ieșire) și GND (masă), iar carcasa sa de tip DDPAK/TO-

263-3 permite montarea pe suprafață și disiparea eficientă a căldurii prin zona de cupru plasată dedesubt.

Pe linia de intrare, tensiunea de +12V este aplicată prin pinul IN și filtrată de condensatorul C1 (10 μ F, 25V), care are rolul de a amortiza eventualele fluctuații sau zgomote de alimentare. În paralel, pe ieșirea pinului OUT este conectat un grup de condensatori: C6 (10 μ F, 10V) și doi condensatori ceramici C7, C8 (100nF, 50V). Aceștia asigură o filtrare corespunzătoare a tensiunii de +5V, reducând ripple-ul și îmbunătățind stabilitatea în regim de sarcină variabilă.

Valoarea tensiunii de ieșire este fixă, stabilizatorul LM1085ISX-5.0 fiind un model preconfigurat pentru 5V, fără a necesita rezistențe de ajustare. Acest avantaj simplifică implementarea și reduce erorile de proiectare în comparație cu versiunile ajustabile. De asemenea, regulatorul oferă protecție internă la supracurent și supratemperatură, contribuind la creșterea fiabilității întregului sistem.

Tensiunea de +5V obținută la ieșirea acestui bloc este apoi direcționată către următorul stabilizator, care va furniza +3,3V către microcontrolerul ESP32 și alte componente logice. Astfel, acest bloc reprezintă o etapă intermediară esențială în lanțul de alimentare al plăcii PCB.

Prin utilizarea unui regulator liniar în acest punct al lanțului, se asigură o tensiune curată și lipsită de zgomot, ideală pentru aplicații sensibile din punct de vedere electric, cum ar fi circuitele de comunicație și componentele logice.

4.3.5 Blocul de conversie 5V la 3.3V

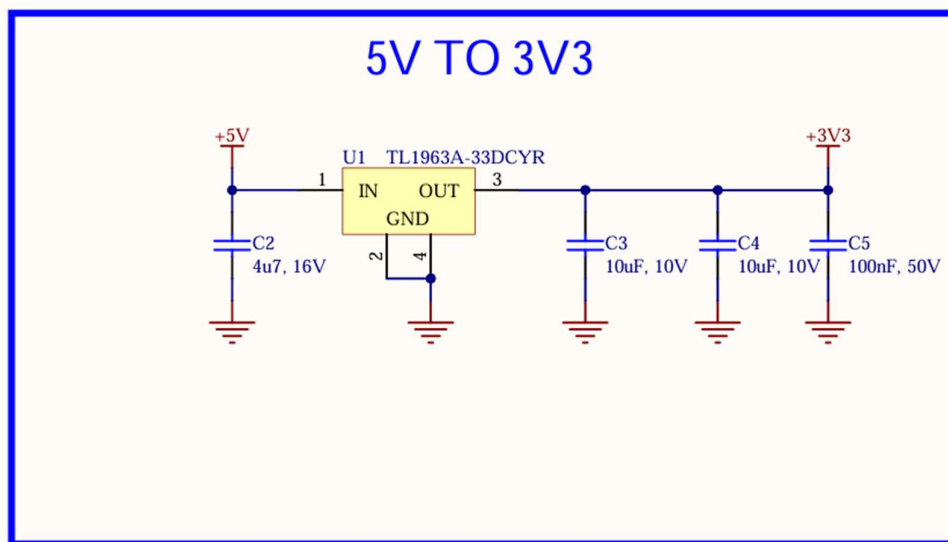


Figura 4.14 - Blocul de conversie 5V la 3.3V

După stabilizarea tensiunii de +5V în blocul anterior, următoarea etapă în lanțul de alimentare o constituie conversia la +3,3V, tensiunea necesară microcontrolerului ESP32 și altor componente logice de joasă tensiune. Acest rol este îndeplinit de un regulator liniar de înaltă performanță, TL1963A-33DCYR, a cărui schemă este ilustrată în Figura 4.14.

Stabilizatorul U1 este un LDO (Low Dropout Regulator), capabil să furnizeze până la 1,5A curent de ieșire, fiind ideal pentru aplicații embedded unde stabilitatea tensiunii este esențială. În această configurație, regulatorul are o tensiune de ieșire fixă de +3,3V, fiind alimentat la intrare cu +5V proveniți de la stabilizatorul precedent (LM1085).

Pentru asigurarea unei alimentări stabile, regulatorul este însoțit de un set de condensatori de filtrare:

- C2 (4.7 μ F, 16V) este plasat la intrare pentru a elimina eventualele fluctuații din linia de 5V;
- C3 și C4 (10 μ F, 10V) sunt conectați în paralel la ieșire și ajută la reducerea ripple-ului și menținerea unei tensiuni constante chiar și în condiții de sarcină variabilă;
- C5 (100nF, 50V) este un condensator ceramic care filtrează componentele de frecvență înaltă și îmbunătățește răspunsul la tranzenți.

Prin această combinație de capacitive, se obține o alimentare extrem de curată și stabilă, esențială pentru funcționarea corectă a microcontrolerului și a comunicațiilor digitale sensibile.

Stabilizatorul TL1963A include și protecții interne la scurtcircuit, supracurent și supratemperatură, oferind un plus de fiabilitate în utilizare. Ieșirea sa este conectată la toate componentele digitale alimentate la 3.3V, inclusiv ESP32-S3-MINI-1-N8 și LED-urile de stare.

Astfel, acest bloc de conversie joacă un rol critic în arhitectura de alimentare a plăcii PCB, asigurând o tensiune precisă, stabilă și sigură pentru funcționarea întregii logicii digitale.

4.3.6 Blocul de interfață USB-C

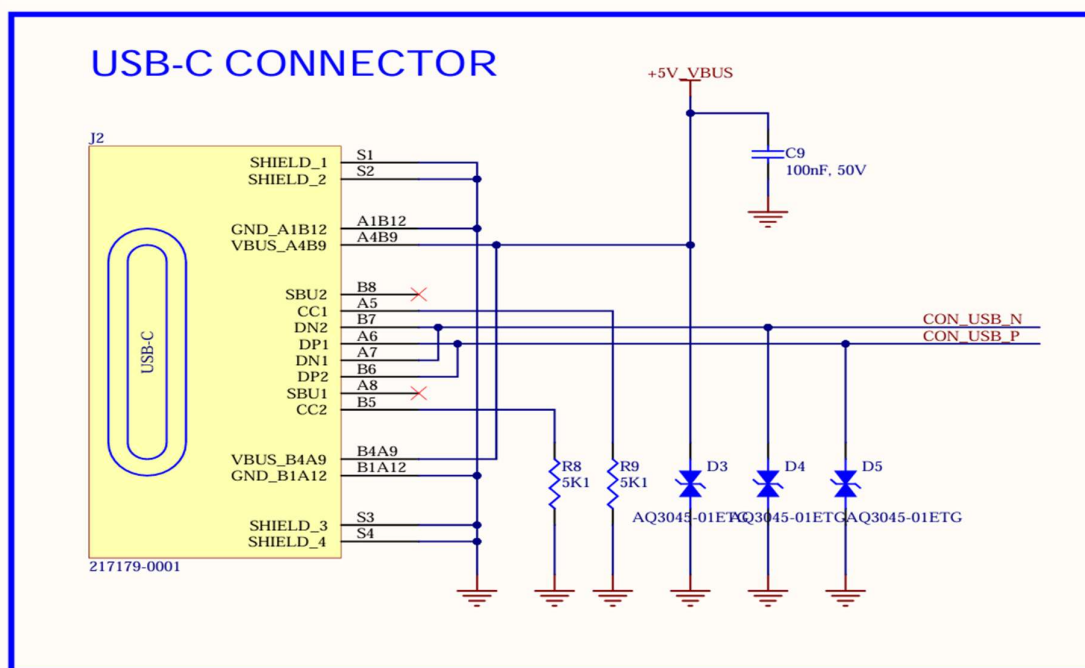


Figura 4.15 - Blocul de interfață USB-C

Blocul de interfață USB-C, ilustrat în Figura 4.15, reprezintă poarta principală de comunicație și alimentare între placa PCB și dispozitivele externe, precum un PC. Acesta are rolul de a prelua semnalele USB și tensiunea VBUS de la conectorul USB-C și de a le transmite către restul circuitelor, protejând în același timp liniile sensibile.

Conectorul utilizat este J2 (217179-0001), un model USB-C standard, care permite conectarea cablurilor USB tip C reversibile. Pinii de date diferențiale D+ (A6) și D- (A7) sunt conectați către

ieșirile CON_USB_N și CON_USB_P, ce duc mai departe semnalul către convertorul USB–UART (FT231XQ-R) din blocul următor.

Pentru protecția liniilor de date, schema integrează trei diode TVS AQ3045-01ETG (D3, D4, D5), conectate la masă. Aceste diode protejează liniile diferențiale USB împotriva descărcărilor electrostatice și supratensiunilor tranzitorii, acționând ca o barieră de protecție împotriva deteriorării circuitelor downstream.

Liniile de date sunt trase prin două rezistențe de 27Ω (R5, R6), montate în serie cu D+ și D-, având rol de potrivire de impedanță și reducere a reflexiilor semnalelor la viteze mari de transfer.

Pe linia de alimentare +5V_VBUS, tensiunea de alimentare provenită din portul USB este filtrată de un condensator ceramic C9 (100nF, 50V), care elimină zgomotele de înaltă frecvență, stabilizând semnalul de alimentare.

Schema include și liniile de control CC1 și CC2, folosite intern de conectorul USB-C pentru detectarea direcției de conectare și negocierea curentului de încărcare, care sunt conectate la masă prin rezistențe interne din conector.

4.3.7 Blocul de conversie USB–UART

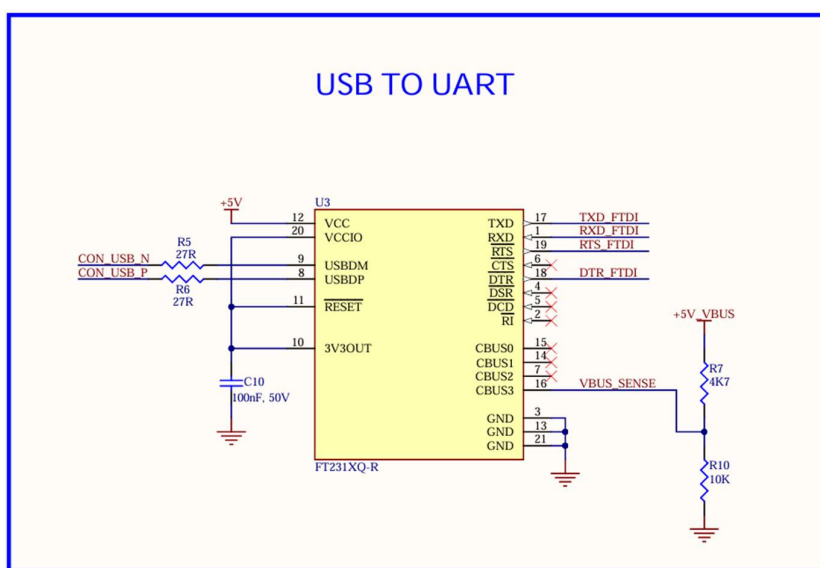


Figura 4.16 - Blocul de conversie USB–UART

Blocul de conversie USB–UART, prezentat în Figura 4.16, are rolul esențial de a transforma semnalele USB primite prin conectorul USB-C în semnale UART compatibile cu microcontrolerul ESP32. Acest bloc facilitează astfel comunicarea serială între placa PCB și un computer sau alt dispozitiv host USB, fiind esențial pentru programare și depanare.

Componenta centrală a acestui bloc este integratul FT231XQ-R (U3), un convertor USB–UART cu montaj pe suprafață, configurat în schema de bază recomandată de producător. Alimentarea acestuia se face cu +5V, aplicată pe pinul VCC (12) și pe pinul VCCIO (20), asigurând compatibilitatea logică la nivel de semnal.

Pe partea de intrare USB, semnalele diferențiale CON_USB_N și CON_USB_P sosesc de la blocul USB-C anterior, fiind trase prin rezistențe de 27Ω (R5, R6), care oferă potrivirea de impedanță

necesară pentru comunicațiile USB. Semnalele sunt conectate la pinii USBDM (9) și USBDP (8) ai FT231XQ-R. Un condensator ceramic C10 (100nF, 50V) este conectat între pinul 3V3OUT (10) și masă, stabilizând tensiunea de ieșire internă a regulatorului încorporat al circuitului.

La ieșirea blocului sunt generate semnalele UART:

- TXD_FTDI (17): transmisia de date de la FTDI către microcontroler;
- RXD_FTDI (1): recepția de date de la microcontroler către FTDI;
- RTS_FTDI (19) și DTR_FTDI (18): semnale de control pentru gestionarea resetării și intrării în modul boot al ESP32.

Linia de detectare a alimentării USB, VBUS_SENSE (16), este conectată la un divizor de tensiune realizat din rezistențele R7 (4.7kΩ) și R10 (10kΩ), permițând integratului FT231XQ-R să detecteze prezența tensiunii VBUS și să activeze comunicația doar când placa este conectată la un dispozitiv host USB.

Pinul RESET (11) rămâne neutilizat, fiind lăsat flotant conform specificațiilor, iar liniile de semnal de control suplimentare (CTS, DSR, DCD, RI, CBUS0–CBUS3) sunt de asemenea neconectate în această aplicație, pentru simplificare.

4.3.8 Blocul de control al semnalelor seriale (Serial Signals Handling)

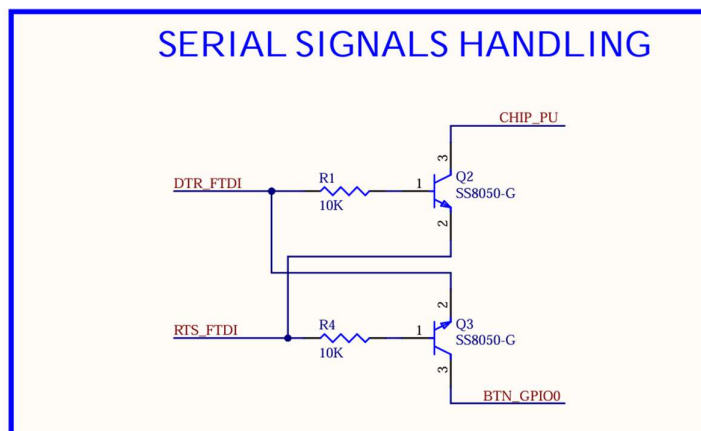


Figura 4.17 - Blocul de control al semnalelor seriale

Blocul de control al semnalelor seriale, prezentat în Figura 4.17, are rolul de a automatiza procesul de inițializare a microcontrolerului ESP32, în special în timpul încărcării firmware-ului prin USB. Acest bloc permite resetarea și intrarea în modul de boot fără intervenție manuală, utilizând semnalele de control provenite de la convertorul USB–UART.

Semnalele DTR_FTDI și RTS_FTDI, generate de integratul FT231XQ-R, sunt utilizate pentru a controla doi tranzistori NPN de tip SS8050-G (Q2 și Q3). Acești tranzistori sunt configurați ca întrerupătoare comandate electronic, iar bazele lor sunt conectate la semnalele FTDI prin intermediul rezistențelor R1 și R4, ambele având valoarea de 10kΩ.

- Tranzistorul Q2 controlează linia CHIP_PU, responsabilă cu resetarea generală a microcontrolerului ESP32. Când semnalul DTR_FTDI este activ (la nivel logic scăzut), tranzistorul Q2 se deschide, trăgând linia CHIP_PU la masă și declanșând o resetare a microcontrolerului.

Restul pinilor GPIO sunt expuși prin conectori suplimentari (CON_ESP_USB_N, CON_ESP_USB_P) sau utilizați intern, în funcție de necesitățile aplicației. Fiecare pin este etichetat conform funcțiilor multiple disponibile pe ESP32, inclusiv ADC, DAC, SPI, I2C sau UART suplimentare.

Schema include și conectarea pinilor critici EN (45) și GPIO46 (44) la semnale de control și la masa sistemului, asigurând funcționarea corectă la pornire și inițializare.

4.3.10 Blocul conectorilor de extensie (Headers)

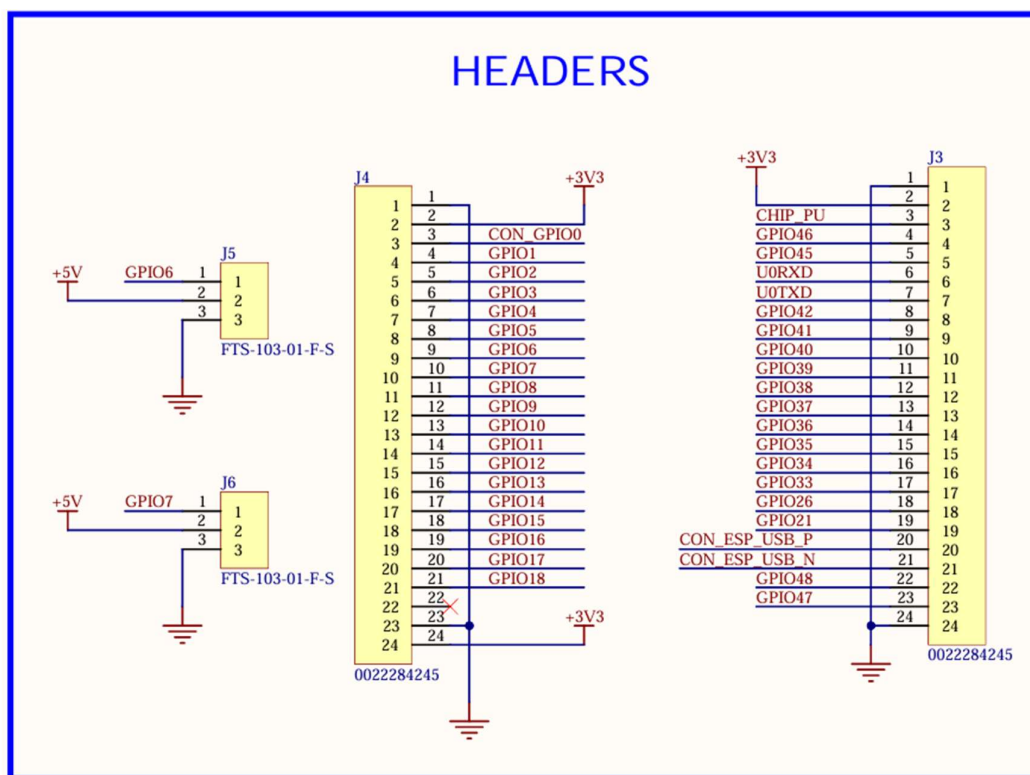


Figura 4.19 – Blocul conectorilor de extensie

Blocul conectorilor externi, ilustrat în Figura 4.19, oferă acces direct la liniile GPIO și la semnalele critice ale microcontrolerului, permițând astfel extinderea funcționalității plăcii PCB prin conectarea unor module periferice suplimentare, senzori sau actuatori.

Schema include trei conectori principali:

- J4 – un header de 24 de pini, care expune majoritatea pinilor GPIO ai microcontrolerului;
- J3 – un al doilea header de 24 de pini, destinat semnalelor speciale, inclusiv liniile de comunicație serială, semnalele de boot și reset, precum și GPIO suplimentare;
- J5 și J6 – doi conectori cu 3 pini fiecare, dedicați semnalelor GPIO6 și GPIO7, fiecare furnizând și o linie de +5V și masă, utilă pentru conectarea unor dispozitive cu consum redus.

Conectorul J4 este conceput ca o interfață generală de extensie, punând la dispoziție pinii GPIO0 – GPIO18, precum și semnalul CON_GPIO0 și o linie de alimentare +3,3V. Acest header

permite accesul rapid la funcțiile principale de intrare/ieșire digitală ale ESP32, inclusiv canale ADC, PWM, I2C sau SPI, în funcție de configurarea firmware-ului.

Conectorul J3 oferă acces la semnalele de control esențiale ale microcontrolerului, precum CHIP_PU (reset), GPIO46, GPIO45, liniile de comunicație U0RXD/U0TXD, dar și GPIO suplimentare (GPIO42 – GPIO48). În plus, acest header expune liniile diferențiale CON_ESP_USB_P și CON_ESP_USB_N, utilizabile pentru comunicații USB suplimentare sau debug.

Conectorii J5 și J6 sunt prevăzuți cu câte un pin de semnal (GPIO6, respectiv GPIO7), o linie de alimentare +5V și o linie de masă, fiind gândiți pentru conectarea rapidă a unor module simple, cum ar fi senzori digitali sau drivere de motoare care necesită alimentare directă din sursa de 5V.

4.3.11 Blocul butoanelor de control manual (RESET și BOOT)

Placa PCB integrează două butoane dedicate controlului manual al microcontrolerului ESP32: unul pentru resetare (RESET) și unul pentru activarea modului de programare (BOOT). Aceste circuite sunt ilustrate în Figura 4.3.9 (buton RESET) și Figura 4.3.10 (buton BOOT), având un rol esențial în procesul de inițializare și depanare.

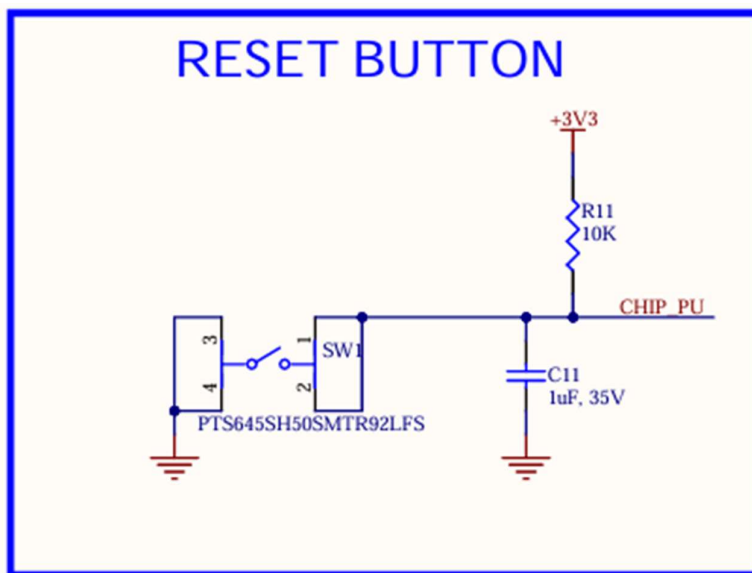


Figura 4.20 - buton RESET

Circuitul pentru resetare, prezentat în Figura 4.20, utilizează un buton tactil SW1 (PTS645SH50SMTR92LFS), conectat între pinul CHIP_PU al ESP32 și masă. În stare normală, linia CHIP_PU este menținută la nivel logic ridicat prin rezistența de 10k Ω (R11) legată la +3,3V. Un condensator de 1 μ F (C11) este montat între linie și masă pentru a filtra zgomotul și a preveni declanșările false.

La apăsarea butonului, linia CHIP_PU este trasă temporar la masă, provocând resetarea microcontrolerului. După eliberarea butonului, rezistența R11 readuce linia la +3,3V, permițând repornirea normală a ESP32. Acest mecanism oferă utilizatorului posibilitatea de a efectua manual o resetare hardware a microcontrolerului, independent de comanda serială.

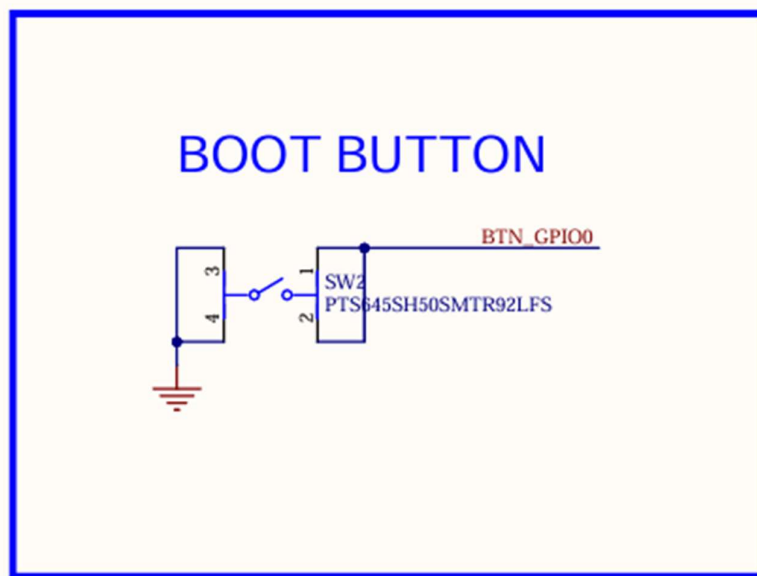


Figura 4.21- buton BOOT

Butonul de boot, ilustrat în Figura 4.21, folosește un comutator identic SW3 (PTS645SH50SMTR92LFS), conectat între linia BTN_GPIO0 și masă. Prin apăsarea acestui buton, pinul GPIO0 al ESP32 este tras la nivel logic scăzut, condiție necesară pentru intrarea în modul de descărcare UART (programare firmware).

Atunci când butonul este apăsat simultan cu resetarea, microcontrolerul intră în modul de bootloader, permițând încărcarea firmware-ului prin portul USB–UART. În lipsa apăsării, pinul GPIO0 rămâne ridicat printr-o rezistență de pull-up externă, iar microcontrolerul pornește normal în aplicația instalată.

4.3.12 Blocul LED-urilor de stare (Power și User)

Placa PCB integrează două LED-uri de stare, cu funcții distincte, ce oferă indicii vizuale privind starea de funcționare a sistemului. Acestea sunt prezentate în Figura 4.22 (LED-ul Power) și Figura 4.23 (LED-ul User).

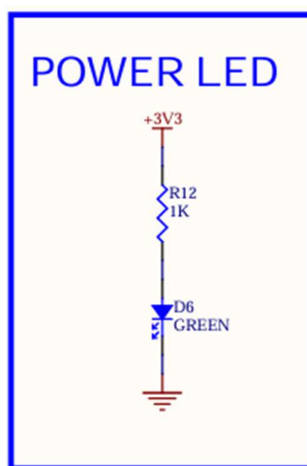


Figura 4.22 - LED-ul Power

Circuitul LED-ului de alimentare, ilustrat în Figura 4.22, este alcătuit dintr-un LED verde D6, conectat în serie cu o rezistență de limitare a curentului R12 (1k Ω). Ansamblul este alimentat direct din linia +3,3V, iar catodul LED-ului este conectat la masă.

Scopul principal al acestui LED este de a semnaliza prezența tensiunii de alimentare pe placa PCB. În momentul în care placa este alimentată corespunzător, LED-ul se aprinde, oferind utilizatorului o confirmare vizuală că stabilizatorul de 3,3V funcționează corect și că sistemul este activ.

Valoarea rezistenței R12 a fost aleasă pentru a limita curentul prin LED la un nivel sigur și suficient pentru o vizibilitate adecvată, prevenind în același timp suprasolicitarea sau deteriorarea LED-ului.

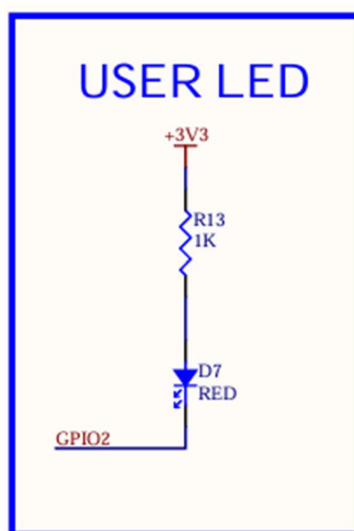


Figura 4.23 - LED-ul User

Circuitul LED-ului de utilizator, prezentat în Figura 4.23, include un LED roșu D7, conectat în serie cu o rezistență de 1k Ω (R13). Spre deosebire de LED-ul Power, anodul acestuia este alimentat din +3,3V, iar catodul este conectat la pinul GPIO2 al microcontrolerului ESP32.

Acest LED este destinat utilizării programabile, permițând aprinderea și stingerea sa din software, prin comenzi trimise către pinul GPIO2. Astfel, dezvoltatorul poate folosi LED-ul User pentru semnalizarea unor evenimente interne, cum ar fi stări de eroare, finalizarea unei sarcini sau indicarea modului activ al aplicației.

În stare inițială, pinul GPIO2 trebuie configurat ca ieșire digitală de tip open-drain sau ca ieșire standard, iar nivelul său logic controlează aprinderea LED-ului. Rezistența R13 are același rol ca în cazul LED-ului Power: limitează curentul prin LED pentru a proteja atât componenta, cât și microcontrolerul.

4.4 Proiectarea, realizarea și fabricarea plăcii de circuit imprimat dedicată modului hardware

4.4.1 Obiectivele proiectării plăcii PCB

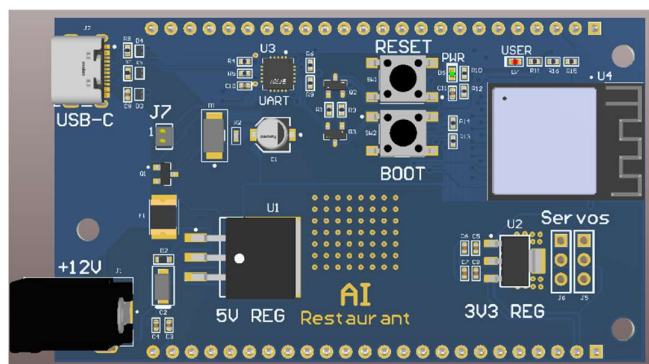


Figura 4.24 - Randare 3D top view PCB

Proiectarea plăcii de circuit imprimat a avut ca obiectiv principal integrarea într-o singură platformă hardware a tuturor funcționalităților necesare sistemului de monitorizare bazat pe inteligență artificială. Această placă a fost concepută pentru a asigura alimentarea corespunzătoare a modulelor electronice, conectivitatea necesară comunicației de date și controlul precis al motorului servomecanic utilizat pentru orientarea camerei.

Un alt obiectiv esențial al proiectării a fost optimizarea traseelor și a stratificării, astfel încât să se minimizeze interferențele electromagnetice și să se asigure stabilitatea semnalelor, mai ales în condițiile unei alimentări mixte la 12V, 5V și 3.3V. În plus, s-a urmărit implementarea unui plan de masă generos, pentru a reduce zgomotul electric și pentru a asigura un potențial de referință uniform pe întreaga suprafață a plăcii.

Placa a fost proiectată pentru a integra un microcontroler ESP32-S3-MINI ca element central, un port de comunicație USB-C cu convertor USB-Serial, circuite de reglare a tensiunii la 5V și 3.3V, butoane de control (RESET și BOOT), LED-uri de stare și conectori dedicați controlului servomotoarelor.

Pe lângă integrarea funcțională, un obiectiv suplimentar a constat în respectarea unui factor de formă redus, astfel încât placa să fie compatibilă cu dimensiunile sistemului mecanic de suport al camerei. În acest sens, s-au utilizat dimensiuni de 3064 mil × 1780 mil, optimizate pentru a încadra toate componentele necesare și a menține accesibilitatea interfețelor de conectare.



Figura 4.25 - Randare 3D bottom view

4.4.2 Specificații tehnice și parametri electrici

Placa de circuit imprimat realizată în cadrul acestui proiect prezintă o serie de caracteristici tehnice atent definite, astfel încât să răspundă cerințelor funcționale și structurale impuse de sistemul hardware. Dimensiunile acesteia sunt de 3064 mil × 1780 mil, rezultând o placă compactă, dar suficient de încăpătoare pentru a găzdui toate componentele necesare. Conform fișei tehnice generate în mediul de proiectare, pe placă au fost montate 51 de componente electronice, dintre care unele în tehnologie SMD, iar altele în montaj THT, în funcție de specificațiile fiecărui element.

Placa este alcătuită din patru straturi conductive, fiecare având un rol bine stabilit în schema electrică și topologia circuitului. Structura stratificată va fi detaliată în subcapitolul următor. În ceea ce privește rutarea, aceasta a fost complet realizată, obținându-se o rată de 100% a conexiunilor rutate, respectiv 188 de conexiuni finalizate, fără legături rămase neconectate.

Analiza traseelor arată o distribuție variată a lățimilor, în funcție de curentul nominal și de funcția fiecărei linii. Cele mai frecvent utilizate lățimi de traseu au fost de 6 mil (pentru semnale de date și GPIO), 10 mil și 20 mil (pentru trasee de putere secundară), respectiv 30 mil și 50 mil pentru traseele principale de alimentare la +12V și +5V, după cum se evidențiază în Anexa 2, care ilustrează Layer 1 al plăcii. Această selecție de lățimi a fost realizată pentru a respecta criteriile de siguranță electrică și pentru a limita căderile de tensiune pe traseele de curent ridicat.

În ceea ce privește distribuția cuprului pe fiecare strat, valorile obținute din analiza layout-ului indică o acoperire de 62% pe Layer 1, 84% pe Layer 2, 89% pe Layer 3, respectiv 26% pe Layer 4. Această distribuție reflectă strategia utilizată în proiectare, unde Layer 3, reprezentând planul de masă, deține cea mai mare suprafață de cupru continuu, așa cum se observă în Anexa 2, contribuind astfel la stabilitatea electrică generală a plăcii și la reducerea zgomotului electromagnetic.

Placa conține 188 de via-uri între straturi, asigurând interconectarea semnalelor verticale între traseele amplasate pe diferite nivele. Aceste via-uri au un diametru standard de 12 mil, corespunzând parametrilor de fabricație acceptați de furnizorul JLCPCB. De asemenea, s-au utilizat 267 de pad-uri pentru componente, incluzând atât pad-uri pentru montaj superficial, cât și pad-uri THT, evidențiate în Figura 4.24 și Figura 4.25, care prezintă randările 3D ale plăcii pe partea superioară și inferioară.

Mai mult, placa integrează 4 sloturi placate și 3 găuri pătrate placate, acestea fiind incluse pentru fixarea mecanică și pentru compatibilitatea cu ansamblul mecanic de prindere. Din punct de vedere al densității traseelor, pe întregul circuit au fost trasate 1582 de piste electrice, cu lățimi variate, iar numărul total de texte și inscripții serigrafice se ridică la 177, incluzând marcaje de identificare a componentelor și simboluri funcționale.

Aceste specificații tehnice confirmă complexitatea moderată a plăcii și demonstrează respectarea standardelor de proiectare pentru un sistem embedded cu multiple surse de alimentare și funcționalități integrate.

4.4.3 Descrierea stratificată a plăcii (stack-up)

Placa de circuit imprimat concepută pentru acest proiect este construită pe o arhitectură cu patru straturi conductive, fiecare având o funcție bine definită în distribuția traseelor electrice. Alegerea unei structuri multilayer a fost determinată de nevoia de a separa logic și fizic traseele de alimentare, semnalele de mare viteză și conexiunile de control, asigurând o performanță electrică superioară și o stabilitate funcțională optimă.

Primul strat, Layer 1, ilustrat în Anexa 2, este utilizat pentru rutarea liniilor principale de alimentare la +12V și +5V, precum și a semnalelor de comunicație USB. Traseele diferențiale dintre portul USB-C și convertorul FT231XQ-R au fost plasate strategic pentru a respecta impedențele controlate, fiind rutate paralel și echidistant. Lățimile traseelor variază între 6 mil și 50 mil, în funcție de sarcina electrică transportată.

Layer 2, de culoare galbenă în reprezentările din Anexa 2, a fost rezervat rutărilor suplimentare, inclusiv traseelor alimentate la +3,3V, și semnalelor auxiliare. Acest strat a completat rutările neacoperite de Layer 1, menținând o distanțare optimă față de planul de masă.

Layer 3, colorat în verde, constituie planul complet de masă al plăcii și oferă o referință electrică comună pentru toate semnalele. Având un grad de acoperire de 89%, acest strat contribuie semnificativ la reducerea zgomotului electromagnetic, fapt evidențiat în imaginile din Anexa 2.

Stratul inferior, Layer 4, reprezentat în albastru, găzduiește traseele GPIO ale microcontrolerului și semnalele de comandă către periferice. Distribuirea acestor trasee în partea inferioară a permis degajarea zonelor superioare pentru alimentare și comunicație. Valorile detaliate de acoperire și trasee sunt analizate și în Anexa 3, prin fișierele Gerber corespunzătoare fiecărui strat.

4.4.4. Straturile auxiliare ale plăcii de circuit imprimat

Pe lângă cele patru straturi conductive, designul plăcii include o serie de straturi auxiliare care nu transportă curent, dar au un rol esențial în procesul de fabricație și montaj. Acestea sunt evidențiate în Anexa 2, unde pot fi observate individual în vizualizările CAD, și detaliate tehnic în Anexa 3, prin fișierele Gerber exportate.

Conturul mecanic al plăcii (Board Shape), desenat cu violet, definește limitele fizice ale plăcii și locațiile sloturilor mecanice. Acesta servește ca ghid pentru procesul de frezare industrială și asigură compatibilitatea cu sistemul mecanic de susținere al camerei. În aceeași anexă se regăsesc și straturile de serigrafie (Legend), atât pentru fața superioară cât și pentru cea inferioară, în care sunt marcate pozițiile componentelor și identificatorii utili în etapa de montaj.

Straturile de pastă de lipit (Paste) sunt dedicate procesului industrial SMD și definesc zonele unde va fi aplicată pasta de lipit prin șablon. Deși în cadrul acestui proiect montajul a fost realizat manual, aceste straturi asigură compatibilitate cu metodele de producție automatizate și pot fi consultate în Anexa 3.

Un alt strat important este masca de lipire (Soldermask), aplicată pentru protejarea cuprului expus și pentru a preveni scurtcircuiturile accidentale. Acesta lasă libere doar pad-urile și via-urile, și este vizibil în reprezentările din Anexa 2, unde se observă și un element decorativ – inscripționarea textului „AI Restaurant”.

Fișierele de foraj (Drill files) definesc locațiile găurilor metalizate și nemetalizate, fiind indispensabile pentru montajul componentelor THT și al conectorilor. Ele sunt incluse în Anexa 3, unde fiecare tip de strat auxiliar este prezentat separat și complet.

4.4.5 Procesul de proiectare CAD și verificare

Proiectarea fizică a plăcii a fost realizată integral în mediul Altium Designer, folosind facilitățile avansate ale acestuia pentru schematizare, rutare și verificare. Procesul a debutat cu definirea conexiunilor funcționale în schema electrică, incluzând microcontrolerul ESP32-S3-MINI, portul USB-C, convertorul USB–UART, stabilizatoarele de tensiune și conectorii de extensie.

După finalizarea schematicii, componentele au fost plasate pe layout în funcție de criterii electrice și mecanice. Amplasarea convertorului USB în imediata vecinătate a conectorului USB-C și apropierea reguletoarelor față de consumatorii locali sunt aspecte care pot fi observate în randările 3D din capitolele anterioare și corelate cu Anexa 2, unde Layer 1 și Layer 2 evidențiază traseele principale.

Rutarea a fost realizată cu respectarea cerințelor de impedanță, lățime minimă și distanțare între trasee. Traseele diferențiale pentru USB sunt rutate paralel, iar liniile de alimentare utilizează lățimi între 20 și 50 mil. Distribuția cuprului între straturi indică o acoperire majoritară pe Layer 3, consolidând planul de masă.

Procesul de validare a inclus verificări automate de tip Design Rule Check (DRC), pentru a asigura respectarea regulilor de fabricație. Ulterior, au fost generate fișierele de producție, inclusiv cele Gerber, ilustrate în Anexa 3. Acestea includ toate straturile necesare pentru fabricație, de la cele conductive până la cele auxiliare, oferind o imagine completă asupra designului final.

4.4.6 Fabricarea și asamblarea plăcii

După încheierea procesului de proiectare în Altium Designer și generarea completă a fișierelor necesare, fabricarea fizică a plăcii de circuit imprimat a fost realizată prin serviciul furnizorului JLCPCB. Fișierele Gerber exportate au inclus toate straturile relevante pentru producție – conductoare, mască de lipire, serigrafie, foraj și contur mecanic – respectând specificațiile tehnice al procesului industrial.



Figura 4.26 – PCB livrat de JLCPCB, înainte de asamblare

Alegerea JLCPCB a fost motivată de fiabilitatea sa, de suportul pentru plăci multi-strat și de rapiditatea execuției. Datele transmise au inclus dimensiunea exactă a plăcii (3064 mil × 1780 mil), arhitectura cu patru straturi conductive, grosimea standard a cuprului, precum și specificațiile de finisare pentru pad-uri. Placa livrată s-a remarcat printr-o execuție precisă a găurilor și sloturilor mecanice, calitate ridicată a serigrafiei și aliniere perfectă a straturilor, conform observațiilor din Figura 4.26.

Ulterior, asamblarea componentelor electronice a fost realizată manual în laborator, începând cu montarea celor în tehnologie SMD (Surface-Mount Device). Întregul proces a fost documentat fotografic în Anexa 4 și a cuprins următoarele etape:

1. Poziționarea plăcii pe un suport fix, pentru a preveni deplasările accidentale în timpul aplicării pastei și montajului (Anexa 4, figura 1);
2. Alinierea și fixarea stencilului metalic, realizat din oțel inoxidabil și comandat de la JLCPCB, utilizat pentru aplicarea precisă a pastei de lipit (Anexa 4, figura 2);
3. Aplicarea pastei de fludor, distribuită uniform cu o racletă peste stencil, pentru a acoperi doar pad-urile expuse (Anexa 4, figura 3);
4. Verificarea vizuală a pastei aplicate, pentru a confirma uniformitatea și absența scurgerilor nedorite;
5. Plasarea componentelor SMD, cu ajutorul unei unelte de tip pick-and-place manual, pentru a asigura alinierea corectă pe pad-uri (Anexa 4, figura 4);
6. Introducerea plăcii în cuptorul reflow, unde a fost supusă unui profil termic de aproximativ 3 minute, activând procesul de lipire uniformă prin topirea controlată a pastei (Anexa 4, figura 5);
7. Inspecția finală sub microscop, pentru verificarea calității lipiturilor și alinierea precisă a componentelor (Anexa 4, figura 6).

După finalizarea montajului SMD, au fost asamblate componentele în tehnologie through-hole (THT), prin inserarea și lipirea manuală a acestora:

- Mufa de alimentare de tip jack, destinată alimentării externe a sistemului;
- Pinii de prindere ai portului USB-C;
- Două rânduri de pini male (2×3), folosiți pentru conectarea a două servomotoare;
- Cele două butoane tactile, pentru reset și comandă locală.

Lipirea acestor componente THT a fost efectuată cu letcon reglabil, utilizând sârmă de fludor cu flux integrat, asigurând o fixare mecanică și electrică durabilă.

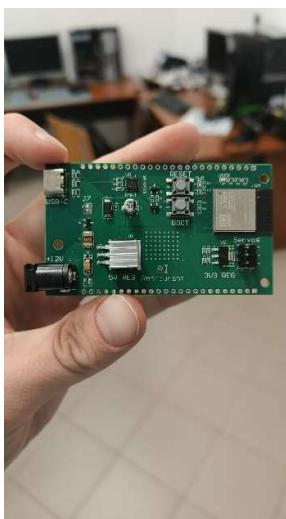


Figura 4.27 - PCB complet asamblat, cu componente SMD și THT

Întregul proces de asamblare a fost realizat cu atenție și precizie, conducând la obținerea unei plăci complet funcționale, pregătită pentru integrarea în sistemul AI de monitorizare. Rezultatul final

este redat în Figura 4.27, unde poate fi observat ansamblul complet al plăcii, cu toate componentele montate.

4.4.7 Testarea plăcii și verificarea funcționalității

După finalizarea procesului de asamblare, placa de circuit imprimat a fost supusă unei serii de teste funcționale și electrice, menite să confirme integritatea conexiunilor, corectitudinea rutării și funcționalitatea componentelor integrate. Testarea a debutat cu o verificare de continuitate a traseelor electrice, utilizând un multimetru digital setat pe modul de detecție a rezistenței. Această etapă a avut ca scop identificarea eventualelor întreruperi sau scurtcircuite apărute accidental în procesul de lipire.

Ulterior, s-a realizat alimentarea treptată a plăcii, inițial aplicând tensiune doar pe liniile de +5V și +3.3V, fără a conecta microcontrolerul la sursa principală, pentru a verifica stabilitatea și corectitudinea tensiunilor de ieșire ale reguletoarelor. Valorile măsurate au confirmat funcționarea adecvată a circuitelor de reglare, în limitele specificate de producătorii acestora.

Un test important a vizat funcționalitatea portului USB-C și a convertorului FT231XQ-R, prin conectarea plăcii la un calculator și verificarea apariției portului serial virtual în managerul de dispozitive al sistemului de operare. Această verificare a demonstrat corectitudinea traseelor diferențiale USB și funcționarea completă a liniei de comunicație.

Funcționarea butoanelor RESET și BOOT a fost testată prin observarea răspunsului microcontrolerului ESP32-S3-MINI la apăsarea acestora, confirmând intrarea corectă în modurile de programare sau resetare. În paralel, LED-urile de semnalizare au fost verificate prin comenzi software trimise către liniile GPIO, iar aprinderea acestora a confirmat atât funcționalitatea componentelor, cât și a traseelor aferente, vizibile pe Layer 4 din Anexa 2.

Un test esențial în validarea funcțională a plăcii PCB a constat în evaluarea ieșirilor dedicate controlului servomotoarelor. Acestea au fost conectate la un servomotor MG996R real, iar răspunsul acestuia a fost analizat în raport cu semnalele PWM generate de microcontrolerul ESP32-S3-MINI, prin intermediul bibliotecii software ESP32Servo. Comportamentul corespunzător al servomotorului a confirmat nu doar acuratețea generării semnalelor de control, ci și calitatea rutării traseelor pe PCB și fiabilitatea interconectărilor dintre microcontroler și conectorii de extensie.

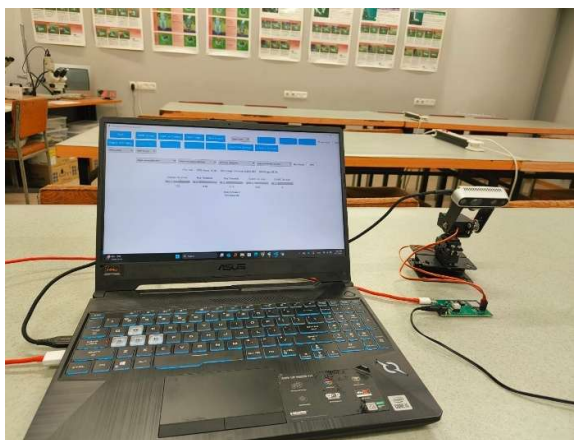


Figura 4.28 – Testul final al plăcii în configurația completă a sistemului

La final, placa a fost supusă unui test integrat, conectând toate modulele hardware în configurația finală a sistemului de monitorizare. Acest test a permis evaluarea funcționării simultane

a tuturor componentelor, inclusiv alimentarea multi-tensiune, comunicația USB, controlul servomotorului și funcționarea LED-urilor de semnalizare, după cum se poate observa în Figura 4.28.

Rezultatele testării au confirmat că placa respectă specificațiile proiectului, oferind stabilitate, funcționalitate completă și compatibilitate cu ansamblul mecanic al sistemului. Nu au fost identificate defecțiuni majore, iar eventualele corecții minore au fost realizate prin retribaj manual, utilizând letconul pentru îmbunătățirea unor îmbinări de lipire.

4.4.8 Concluzii asupra procesului de realizare

Procesul de proiectare, fabricare și asamblare a plăcii de circuit imprimat a reprezentat o etapă esențială în realizarea modului hardware al sistemului de monitorizare. Prin aplicarea unui flux riguros de proiectare, utilizând platforma Altium Designer, s-a obținut un design optimizat din punct de vedere electric, mecanic și funcțional, capabil să integreze toate modulele necesare într-o arhitectură compactă și fiabilă.

Fabricarea plăcii la furnizorul JLCPCB a demonstrat compatibilitatea fișierelor generate cu cerințele de producție, iar calitatea execuției a permis asamblarea manuală fără dificultăți majore. Componentele selectate, provenite de la furnizori consacrați precum DigiKey și LCSC, au asigurat fiabilitatea necesară funcționării pe termen lung, iar procesul de montaj, realizat manual în laborator, a permis un control direct al calității fiecărei etape.

Testele funcționale au confirmat că placa respectă toate specificațiile proiectului, incluzând stabilitatea tensiunilor de alimentare, funcționalitatea comunicației USB, răspunsul corect al liniilor GPIO și controlul precis al servomotoarelor. Nu au fost identificate defecțiuni structurale sau electrice majore, iar eventualele mici corecții au putut fi remediate prin intervenții punctuale, fără a afecta funcționarea globală a plăcii.

Capitolul 5: Implementarea software

5.1 Arhitectura generală a aplicației

5.1.1 Organizarea fișierelor și structura modulară a proiectului

Aplicația software dezvoltată în cadrul acestui proiect este structurată pe o arhitectură modulară clar definită, construită pe principiile separării responsabilităților și scalabilității. Proiectul este organizat în directorul principal „AI_restaurant”, care cuprinde directoare esențiale pentru funcționarea completă a sistemului: `data`, `docs`, `src`, `static` și `templates`, alături de fișierul principal `main.py`, care acționează ca punct de pornire al aplicației.

Directorul `src` reprezintă nucleul logic al aplicației, incluzând toate modulele software majore. Acesta este împărțit în următoarele componente:

- **detectors/** – conține fișierul `yolo_detector.py`, responsabil pentru implementarea logicii de detecție AI utilizând modelele YOLO.
- **gui/** – gestionează interfața grafică a aplicației, incluzând subfoldere precum `frames` (pentru componente vizuale reutilizabile), `modes` (pentru modurile de operare: captură live, video, imagine) și fișierele principale `admin_gui.py` și `user_gui.py`.
- **managers/** – administrează logica asociată entităților principale (mese, persoane, alarme) prin fișiere precum `table_manager.py`, `people_manager.py` și `alarm_manager.py`.
- **reports/** – include modulele `automatic_report_generator.py` și `manual_report_generator.py`, dedicate generării automate sau manuale a rapoartelor, integrând informațiile colectate în cadrul aplicației într-un format accesibil și structurat.
- **utils/** – reunește funcții auxiliare precum `correct_illumination.py`, `detection_utils.py`, `time_utils.py` și `excel_utils.py`, care sprijină procesarea imaginilor, gestionarea timpilor și crearea rapoartelor.
- **web/** – oferă o bază pentru extinderea aplicației către o interfață web viitoare.
- **libs.py** – centralizează importurile esențiale utilizate la nivel global în aplicație.

Această organizare modulară permite dezvoltarea, testarea și extinderea eficientă a aplicației, facilitând în același timp înțelegerea clară a fluxurilor interne și a interdependențelor dintre module.

5.1.2 Fluxul funcțional al aplicației

Fluxul logic al aplicației urmează o succesiune bine definită de etape:

1. **Capturarea imaginii** – utilizatorul selectează sursa video (cameră web, fișier video sau imagine). Acest pas este gestionat prin interfața grafică și modulele din `gui/modes`.
2. **Preprocesarea imaginii** – cadrul video este transmis către funcția `display_frame`, unde, dacă este activată, se aplică funcția `correct_illumination` pentru ajustarea iluminării pe baza analizei histogramei și a luminozității. Această etapă sporește precizia detecțiilor în condiții de lumină insuficientă sau excesivă.

3. **Detectția obiectelor** – cadrul preprocesat este introdus în modelul YOLO (v8, v9 sau v10), care returnează o listă cu obiectele detectate, inclusiv persoane (clasa 0), mese (clasa 60) și obiecte roșii (ex. sticle, farfurii).

4. **Filtrare și analiză geometrică** – obiectele detectate sunt procesate prin `filter_detections` pentru a elimina suprapunerile nedorite, iar ulterior sunt analizate prin `table_status.py` pentru determinarea stării fiecărei mese, pe baza clasei, poziției și ariei de suprapunere (IoU).

5. **Afișarea rezultatelor** – imaginea rezultată, cu detecțiile etichetate, este afișată în GUI, împreună cu scorurile de centrare și starea actuală a fiecărei mese (exemplu: „Table 2: eating for 00:03:45”).

6. **Acțiuni hardware și export date** – în cazul în care este detectată o stare critică (de exemplu, timp excesiv la masă, număr prea mare de persoane), se declanșează alarme sonore. Datele sunt salvate în fișiere `.json`, iar programul poate genera rapoarte `.xlsx` utilizând modulele din `reports/`, respectiv `automatic_report_generator.py` sau `manual_report_generator.py`.

7. **Controlul camerei video** – comenzile de mișcare a camerei sunt transmise prin USB-C (protocol UART logic) către microcontrollerul ESP32, care controlează două servomotoare. Această funcționalitate este integrată în interfața `move_camera_frame.py`.

5.1.3 Tehnologii utilizate

Proiectul este implementat în principal în Python 3.12 și integrează următoarele biblioteci și tehnologii:

- **YOLOv8/v9/v10 (Ultralytics)** – utilizate pentru detecția rapidă și precisă a obiectelor, cu modele preantrenate.
- **OpenCV** – pentru manipularea cadrelor video și prelucrarea imaginilor.
- **Tkinter și ttk** – pentru crearea interfeței grafice desktop.
- **NumPy și Pandas** – pentru operații numerice și manipulare de date.
- **Matplotlib** – pentru vizualizarea statisticilor.
- **openpyxl** – pentru crearea și manipularea fișierelor Excel (`.xlsx`).
- **pyserial** – pentru comunicarea UART cu microcontrollerul ESP32.
- **json și datetime** – pentru gestionarea fișierelor de configurare și a marcajelor temporale.

Această combinație de tehnologii asigură:

- compatibilitate cu sisteme de operare moderne,
- flexibilitate pentru extinderi viitoare (exemplu: trecere la aplicație web),
- suport activ din partea comunității,
- performanță ridicată în procesarea imaginilor și generarea rapoartelor.

De asemenea, pentru microcontrollerul ESP32, codul este dezvoltat în C++ utilizând platforma Arduino IDE, iar comunicarea serială este gestionată asincron, permițând interpretarea comenzilor în timp real – aspect detaliat în subcapitolul 5.5.

5.2 Implementarea detecției obiectelor folosind YOLO

5.2.1 Motivația alegerii algoritmului YOLO

În cadrul acestui proiect, algoritmul YOLO (You Only Look Once) a fost ales pentru realizarea detecției obiectelor, datorită vitezei ridicate și acurateței bune, obținute într-un singur pas

de procesare. Acest algoritm aparține clasei de modele de detecție „single-stage”, ceea ce înseamnă că predicțiile sunt generate direct, fără a necesita pași suplimentari de filtrare sau clasificare, facilitând procesarea în timp real. Aceste caracteristici îl fac potrivit chiar și pentru sisteme hardware cu resurse limitate.

Pe lângă rapiditate, YOLO se remarcă prin adaptabilitate. În aplicația realizată, au fost utilizate modele din generațiile YOLOv8, YOLOv9 și YOLOv10, configurate cu greutatea pre-antrenate, astfel încât să recunoască eficient obiectele relevante: persoane, mese și obiecte de culoare roșie, cum ar fi sticle sau farfurii. Alegerea acestui algoritm a permis integrarea rapidă și precisă a unui sistem de recunoaștere a obiectelor fără necesitatea unui antrenament suplimentar pe date personalizate.

5.2.2 Structura detectorului YOLO (fișierul `yolo_detector.py`)

Fișierul `yolo_detector.py`, aflat în directorul `src/detectors`, definește o clasă numită `YoloDetector`. Aceasta gestionează încărcarea modelului YOLO, prelucrarea imaginilor, realizarea predicțiilor și vizualizarea rezultatelor.

Inițializarea clasei implică identificarea automată a resursei hardware disponibile (CPU sau GPU), stabilirea căii către model și apelarea metodei de încărcare a acestuia:

```
class YoloDetector:
    def __init__(self, model_path="models/yolov10l.pt"):
        self.device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")

        self.model_path = model_path
        self.load_model(self.model_path)
```

Modelul YOLO este încărcat în memoria dispozitivului selectat prin metoda `load_model`, utilizând biblioteca Ultralytics:

```
def load_model(self, model_path):
    self.model = YOLO(model_path, verbose=False).to(self.device)
```

5.2.3 Prelucrarea imaginii – `correct_illumination.py`

Pentru a asigura o detecție eficientă chiar și în condiții de iluminare deficitară sau neuniformă, aplicația integrează o funcție de corectare a iluminării, implementată în fișierul `correct_illumination.py`. Această funcție este apelată din `display_frame.py` atunci când utilizatorul activează opțiunea de ajustare automată.

Funcția combină mai multe tehnici de procesare:

- ajustare adaptivă a contrastului prin algoritmul CLAHE;
- reglarea automată a gamma-ului;
- normalizarea nivelurilor de culoare;
- reducerea selectivă a zgomotului.

Fragmentul de cod ilustrează aplicarea acestor operații:

```
clahe = cv2.createCLAHE(clipLimit=clahe_clip,
tileGridSize=(clahe_tile_size, clahe_tile_size))
l_clahe = clahe.apply(l)
inv_gamma = 1.0 / gamma
```

```
frame_gamma = cv2.LUT(frame_clahe, gamma_table)
```

Această etapă s-a dovedit esențială pentru îmbunătățirea calității imaginilor în spații cu lumină slabă sau reflexii puternice, precum cele întâlnite în mediile de tip restaurant.

5.2.4 Fluxul detecției – de la imagine la etichetare

După procesarea inițială, cadrul este convertit într-un tensor și transmis către modelul YOLO pentru inferență:

```
frame_tensor = torch.from_numpy(frame_normalized).to(device).permute(2, 0, 1).unsqueeze(0)
results = self.model(frame_tensor)
```

Rezultatele includ coordonatele casetelor de delimitare, clasa atribuită și nivelul de încredere. Acestea sunt organizate într-o listă de dicționare, fiecare conținând informațiile asociate unui obiect detectat:

```
detections.append({
    'box': (x1.item(), y1.item(), x2.item(), y2.item()),
    'class': cls,
    'confidence': conf.item()
})
```

5.2.5 Filtrarea rezultatelor și clasificarea stării meselor

După obținerea detecțiilor brute, sistemul aplică o filtrare suplimentară prin funcția `filter_detections`, care elimină suprapunerile redundante utilizând o valoare prag (IoU). Ulterior, informațiile procesate sunt transmise către `table_status.py`, unde este calculată starea fiecărei mese în funcție de obiectele prezente și de poziționarea acestora.

Sistemul clasifică mesele în patru stări posibile:

- available
- ready to order
- eating
- need to clean

Determinarea stării se bazează pe numărul obiectelor roșii și pe prezența persoanelor în zona mesei, utilizând o logică simplă:

```
if red_count > 0 and blue_count == 0:
    return "need to clean"
elif blue_count > 0 and red_count == 0:
    return "ready to order"
```

În această abordare, culorile roșu și albastru sunt folosite ca simboluri semantice:

- **Roșu** – indică prezența obiectelor din clasele 39–55 (ex. farfurii, pahare, tacâmuri, sticle), semnalând resturi pe masă.
- **Albastru** – corespunde clasei 0 (persoană), indicând prezența unui client.

Această metodă permite o evaluare automată și intuitivă a statusului meselor, folosind doar datele vizuale extrase.

5.2.6 Integrarea rezultatelor în interfața grafică

Rezultatele obținute sunt afișate în interfața grafică prin funcțiile `draw_detections`, `draw_auto_detections` și `draw_detection_with_table_id`. Aceste funcții adaugă etichete pe imagine, indicând clasa obiectului și durata stării asociate:

```
cv2.putText(frame, label, (int(x1), int(y1) - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)
```

Funcțiile sunt apelate din `display_frame.py`, în funcție de modul activat de utilizator (captură live, fișier video sau imagine).

5.2.7 Evaluarea performanței – viteză și acuratețe

Performanța aplicației a fost evaluată prin măsurarea numărului de cadre procesate pe secundă (FPS) și prin calcularea scorului de centrare a fiecărei mese. FPS-ul este determinat astfel:

```
current_time = time.time()
delta_time = current_time - self.prev_time
if delta_time > 0:
    fps = 1 / delta_time
```

Pe un sistem echipat cu placă grafică NVIDIA, aplicația a atins viteze între 30 și 60 FPS în modul live, menținând totodată o acuratețe ridicată datorită modelelor YOLO utilizate.

5.2.8 Concluzii și beneficii ale implementării

Integrarea algoritmului YOLO în acest proiect a oferit o serie de avantaje esențiale: procesare rapidă în timp real, clasificare automată a statusului meselor, afișare dinamică în interfața grafică și rezistență la variații de iluminare. Sistemul este ușor de extins pentru a recunoaște și alte tipuri de obiecte, dacă va fi necesar în viitor.

Prin utilizarea funcției de corectare a iluminării, alături de analiza geometrică și integrarea vizuală, aplicația asigură o monitorizare eficientă și precisă a meselor într-un mediu de restaurant. Alegerea algoritmului YOLO s-a dovedit ideală pentru cerințele de performanță și fiabilitate ale proiectului.

În capitolul următor va fi analizat în detaliu modul de gestionare a stării meselor, concentrându-se pe algoritmii geometrici implementați.

5.3 Analiza și clasificarea stării meselor prin geometrie computațională

5.3.1 Introducere

După ce obiectele de interes au fost detectate, un pas esențial în procesul aplicației a fost clasificarea automată a stării fiecărei mese. Această clasificare nu s-a bazat exclusiv pe simpla prezență a obiectelor, ci a integrat concepte din geometria computațională, pentru a evalua poziția relativă a obiectelor detectate față de suprafața fiecărei mese. Astfel, analiza fiecărei mese a fost realizată ținând cont de gradul de suprapunere al obiectelor roșii și al persoanelor cu aria mesei, precum și de proporțiile acestora. În continuare sunt prezentate principiile și algoritmii implementați pentru o clasificare precisă și adaptabilă a stării meselor.

5.3.2 Algoritmi pentru determinarea stării mesei pe baza poziției obiectelor

Clasificarea automată a unei mese se bazează pe trei elemente principale obținute prin procesare geometrică: identificarea obiectelor relevante, calculul ariilor și determinarea gradului de suprapunere (IoU). Această logică este implementată în clasa `TableStatus`, din fișierul `table_status.py`, apelată pentru fiecare masă detectată în timpul funcționării aplicației.

```
class TableStatus:
    def check_table_status(self, table_box, detections):
        red_objects, blue_objects = self.filter_relevant_objects(table_box,
detections)
        table_area = calculate_area(table_box)

        red_count = sum(
            self.is_relevant_object(table_box,    obj['box'],    table_area,
self.red_threshold)
            for obj in red_objects
        )
        blue_count = sum(
            self.is_relevant_object(table_box,    obj['box'],    table_area,
self.blue_threshold)
            for obj in blue_objects
        )

        if red_count > 0 and blue_count == 0:
            return "need to clean"
        elif red_count == 0 and blue_count == 0:
            return "available"
        elif blue_count > 0 and red_count == 0:
            return "ready to order"
        elif blue_count > 0 and red_count > 0:
            return "eating"
        return "unknown"
```

5.3.3 Calculul ariilor și al suprapunerii (IoU)

Un aspect esențial al analizei geometrice constă în calcularea ariei fiecărui obiect detectat și a gradului de suprapunere dintre obiect și masa analizată. Aceste operații sunt efectuate prin funcțiile `calculate_area()` și `calculate_iou()`:

```
def calculate_area(box):
    x1, y1, x2, y2 = box
    return max(0, x2 - x1) * max(0, y2 - y1)

def calculate_iou(box1, box2):
    x1, y1, x2, y2 = box1
    x3, y3, x4, y4 = box2

    x11 = max(x1, x3)
    y11 = max(y1, y3)
    x12 = min(x2, x4)
    y12 = min(y2, y4)

    inter_width = max(0, x12 - x11)
    inter_height = max(0, y12 - y11)
    inter_area = inter_width * inter_height

    box1_area = calculate_area(box1)
    box2_area = calculate_area(box2)

    union_area = box1_area + box2_area - inter_area
```

```
return inter_area / union_area if union_area > 0 else 0
```

5.3.4 Praguri de relevanță și adaptabilitate

Utilizatorul are posibilitatea de a regla valorile pragurilor `red_threshold` și `blue_threshold` direct din interfața grafică, pentru a ajusta sensibilitatea algoritmului în stabilirea relevanței obiectelor. Aceste valori sunt transmise din AdminGUI către instanțele clasei `TableStatus` prin intermediul `TableManager`.

```
def set_red_threshold_for_all_tables(self, threshold):
    for table in self.tables.values():
        table.set_red_threshold(threshold)
```

Această funcționalitate asigură adaptarea sistemului la condiții variabile de iluminare sau aglomerare.

5.3.5 Rolul culorilor în procesul de clasificare

În logica algoritmului, culoarea obiectelor joacă un rol central. Obiectele considerate roșii (clasele 39–55) sunt asociate cu semnele unei mese utilizate, în timp ce clasa 0 (persoană) indică prezența unui client activ.

```
def filter_relevant_objects(self, table_box, detections):
    red_objects, blue_objects = [], []
    for det in detections:
        class_id = det['class']
        if class_id in range(39, 56):
            if calculate_iou(table_box, det['box']) > 0:
                red_objects.append(det)
        elif class_id == 0:
            if calculate_iou(table_box, det['box']) > 0:
                blue_objects.append(det)
    return red_objects, blue_objects
```

5.3.6 Optimizări pentru funcționare în medii reale

Pentru a preveni schimbările bruște ale stării mesei din cauza fluctuațiilor de detecție, a fost implementat un filtru temporal care înregistrează doar acele stări ce persistă un timp minim de o secundă:

```
def update_status(self, new_status):
    current_time = datetime.now()
    duration = (current_time - self.start_time).total_seconds()
    if duration >= 1.0:
        self.status_durations.append((self.current_status, duration))
        self.previous_status = self.current_status
        self.current_status = new_status
        self.start_time = current_time
```

5.3.7 Funcția clasei `TableStatus` în ansamblul aplicației

Fiecare masă este monitorizată printr-o instanță a clasei `TableStatus`, care implementează integral logica de analiză și clasificare. Instanțele sunt gestionate colectiv de `TableManager`, responsabil pentru actualizarea continuă a stării meselor detectate.

5.4 Integrarea pragurilor temporale și generarea rapoartelor de stare

5.4.1 Introducere

Pentru a completa procesul de monitorizare vizuală și a oferi un suport decizional relevant pentru managementul restaurantului, sistemul dezvoltat integrează un mecanism de înregistrare și analiză temporală a stărilor detectate. Această funcționalitate permite nu doar afișarea în timp real a stării fiecărei mese, ci și elaborarea automată a unor rapoarte statistice zilnice, săptămânale sau lunare, utile pentru îmbunătățirea proceselor operaționale.

5.4.2 Implementarea monitorizării temporale a fiecărei mese

Fiecare obiect `TableStatus` conține o variabilă `start_time` și o listă `status_durations`, în care sunt stocate toate tranzițiile de stare și duratele aferente. La fiecare schimbare de stare validată (care persistă peste un prag de 1 secundă), o nouă înregistrare este adăugată în listă:

```
def update_status(self, new_status):
    current_time = datetime.now()
    duration = (current_time - self.start_time).total_seconds()
    if duration >= 1.0:
        self.status_durations.append((self.current_status, duration))
        self.previous_status = self.current_status
        self.current_status = new_status
        self.start_time = current_time
```

Această abordare garantează că doar tranzițiile semnificative sunt luate în considerare și incluse în analiza finală.

5.4.3 Stocarea înregistrărilor în fișiere JSON

Pe baza acestor înregistrări, clasele `TableManager` și `PeopleManager` creează periodic fișiere JSON care conțin informații despre stările meselor și numărul de persoane detectate. Salvarea acestor fișiere se face automat la intervale prestabilite, utilizând un mecanism bazat pe fire de execuție (threading):

```
def auto_save_thread(self):
    while True:
        if self.auto_save_enabled:
            self.save_to_file()
            time.sleep(self.save_interval)
```

Fiecărei mese îi este atribuit un identificator unic, iar fiecare fișier JSON reflectă istoricul complet al stărilor pentru masa respectivă pe durata sesiunii de rulare.

5.4.4 Analiza statistică și generarea rapoartelor în Excel

Fișierele JSON generate sunt procesate prin clasa `JsonToExcel`, care centralizează datele și le exportă într-un fișier Excel. Această funcționalitate este accesibilă utilizatorului prin butonul „Analyze Records” din interfața AdminGUI. Procesul implică obținerea mediei, valorii maxime și duratei totale pentru fiecare stare a mesei:

```
def save_average_statistics(self):
    for table_id in self.table_stats:
        for status, durations in self.table_stats[table_id].items():
            avg = round(statistics.mean(durations), 2) if durations else 0
```



```
self.json_data[table_id][status]['average_duration'] = avg
```

Prin această analiză, utilizatorul poate identifica mesele care rămân necurățate o perioadă prea lungă, poate evalua timpii de așteptare și poate determina gradul de ocupare în funcție de intervalele orare.

5.4.5 Interfața grafică pentru selecția rapoartelor

Sistemul pune la dispoziția utilizatorului o interfață grafică intuitivă pentru selectarea fișierelor JSON ce urmează a fi analizate. În plus, aplicația creează automat directoare date pentru salvarea rapoartelor, asigurând o organizare logică și cronologică a fișierelor generate:

```
current_date = datetime.now().strftime("%Y-%m-%d")
excel_output_dir = os.path.join("data", "outputs", "daily_report",
current_date)
os.makedirs(excel_output_dir, exist_ok=True)
```

Această automatizare reduce riscul de erori umane și simplifică utilizarea aplicației chiar și de către persoane fără pregătire tehnică.

5.4.6 Utilizarea bibliotecilor Python pentru analiză și generare de rapoarte

Funcționalitatea de analiză și generare a rapoartelor este susținută de o serie de biblioteci Python, alese pentru fiabilitatea și popularitatea lor:

- pandas pentru manipularea și procesarea datelor;
- matplotlib pentru crearea graficelor și vizualizărilor;
- openpyxl pentru manipularea fișierelor Excel;
- json și datetime pentru gestionarea fișierelor de intrare și a marcajelor temporale.

Aceste instrumente oferă o bază solidă și flexibilă, fiind recunoscute pentru compatibilitatea lor cu aplicații de analiză a datelor.

5.4.7 Concluzii privind integrarea analizei temporale

Această componentă adaugă sistemului o dimensiune analitică suplimentară, transformând aplicația dintr-un simplu instrument de monitorizare vizuală într-o platformă completă de suport decizional. Utilizarea fișierelor JSON ca bază de date intermediară, împreună cu generarea rapoartelor Excel, oferă un echilibru optim între flexibilitate, ușurință în utilizare și posibilități de extindere viitoare.

5.5 Interfața grafică și rolul utilizatorului în configurarea sistemului

5.5.1 Introducere

Interfața grafică (GUI) dezvoltată în cadrul proiectului reprezintă legătura principală dintre utilizator și sistemul automatizat de monitorizare. Prin această interfață, utilizatorul beneficiază de control complet asupra funcțiilor de detecție, configurare a camerei, analiză statistică și generare de rapoarte. În acest subcapitol sunt prezentate structura interfeței și funcționalitățile disponibile, atât pentru utilizatorii de tip administrator, cât și pentru utilizatorii obișnuiți.

5.5.2 Structura interfeței grafice

Aplicația oferă două niveluri de interacțiune: Admin GUI și User GUI. Admin GUI oferă acces extins la funcții de configurare, testare și analiză, în timp ce User GUI este limitată la rularea aplicației cu parametri predefiniți încărcăți anterior.

Structura Admin GUI include mai multe module și componente vizuale, organizate astfel:

- `move_camera_frame.py` – controlează poziția camerei motorizate;
- `schedule_frame.py` – permite programarea intervalelor orare pentru funcționarea automată;
- `calendar_frame.py` – facilitează selectarea perioadelor pentru analiză statistică;
- `test_servos_frame.py` – oferă posibilitatea de testare individuală a servomotoarelor MG996R;
- `display_frame.py` – afișează în timp real rezultatele detecției YOLO.

5.5.3 Controlul detecției și ajustarea parametrilor

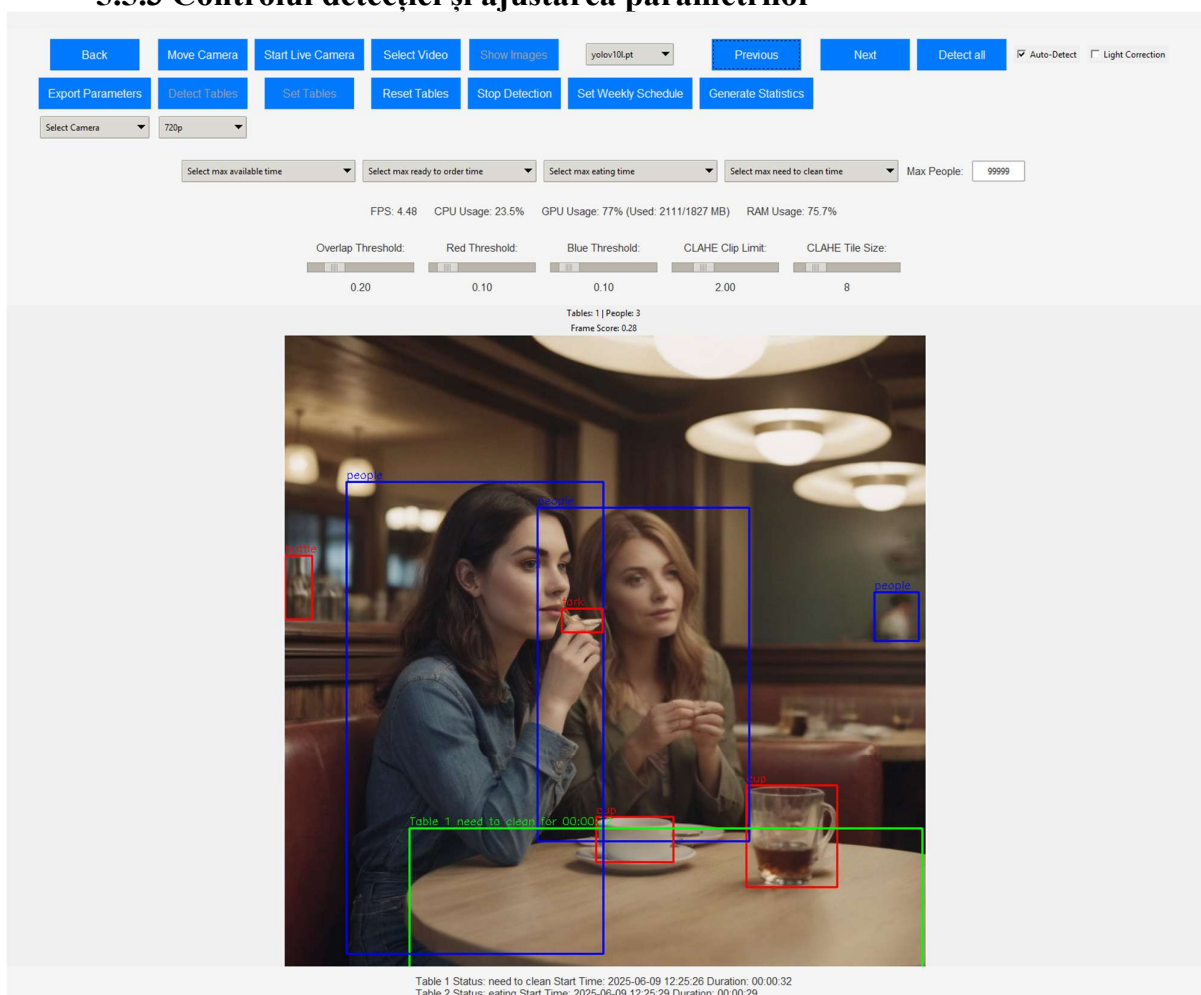


Figura 5.1 – Interfața Admin GUI

În cadrul Admin GUI, după cum este ilustrat în Figura 5.1, utilizatorul are posibilitatea de a selecta modelul YOLO dorit dintr-un meniu derulant, de a alege sursa video (cameră web sau fișier video) și de a activa diferite moduri de detecție, printre care:

- Detectare completă (Detect All);
- Detectare exclusivă a meselor (Detect Tables);
- Setare poziții mese (Set Tables);
- Resetare configurare mese (Reset Tables).

De asemenea, poate fi activat modul Auto-Detect, care rulează automat un algoritm de detecție continuă, actualizând în mod dinamic starea fiecărei mese.

Parametrii de prag, esențiali în clasificarea corectă a stării meselor, pot fi reglați prin glisoare:

- Overlap Threshold – stabilește procentul minim de suprapunere între un obiect și masa analizată pentru ca obiectul să fie considerat relevant;
- Red Threshold și Blue Threshold – ajustează filtrarea obiectelor roșii sau a persoanelor, în funcție de raportul dintre aria obiectului și cea a mesei.

5.5.4 Selectarea camerei și a rezoluției

Prin intermediul interfeței, utilizatorul poate selecta una dintre camerele disponibile (detectate automat de aplicație) și poate alege rezoluția dorită (480p, 720p sau 1080p). Aceste setări permit adaptarea performanței detecției la resursele hardware disponibile.

5.5.5 Configurarea timpilor maximi și a numărului de persoane

Un element important al aplicației este posibilitatea de a seta timpii maximi acceptați pentru fiecare stare posibilă a mesei (available, ready to order, eating, need to clean). Aceste valori pot fi selectate din liste derulante, câte una pentru fiecare stare. În plus, utilizatorul poate introduce un număr maxim admis de persoane într-un cadru, depășirea acestuia declanșând o alertă.

```
self.time_available = tk.StringVar(value='Select Max Time Available')
self.time_ready = tk.StringVar(value='Select Max Time Waiting')
self.time_eating = tk.StringVar(value='Select Max Time Eating')
self.time_clean = tk.StringVar(value='Select Max Time Clean')
```

Aceste setări sunt esențiale pentru logica de declanșare a alertelor și pentru identificarea potențialelor situații anormale.

5.5.6 Salvarea configurației

După finalizarea configurării, aplicația oferă opțiunea de a salva parametrii într-un fișier JSON localizat în directorul `data/config`, identificat printr-un timestamp unic. Acest fișier poate fi ulterior importat în User GUI, facilitând rularea aplicației în regim operațional.

```
config_data = {
    "selected_camera": self.selected_camera,
    "selected_model": self.model_var.get(),
    "video_resolution": self.video_resolution,
    "max_times": {
        "available": self.time_available.get(),
        "ready_to_order": self.time_ready.get(),
        "eating": self.time_eating.get(),
        "need_to_clean": self.time_clean.get()
    },
    "max_people": int(self.max_people_var.get()),
    "overlap_threshold": round(float(self.overlap_threshold_var.get()),
```

2),

```

    "red_threshold": round(float(self.red_threshold_var.get()), 2),
    "blue_threshold": round(float(self.blue_threshold_var.get()), 2),
}

```

5.5.7 Interfața User GUI

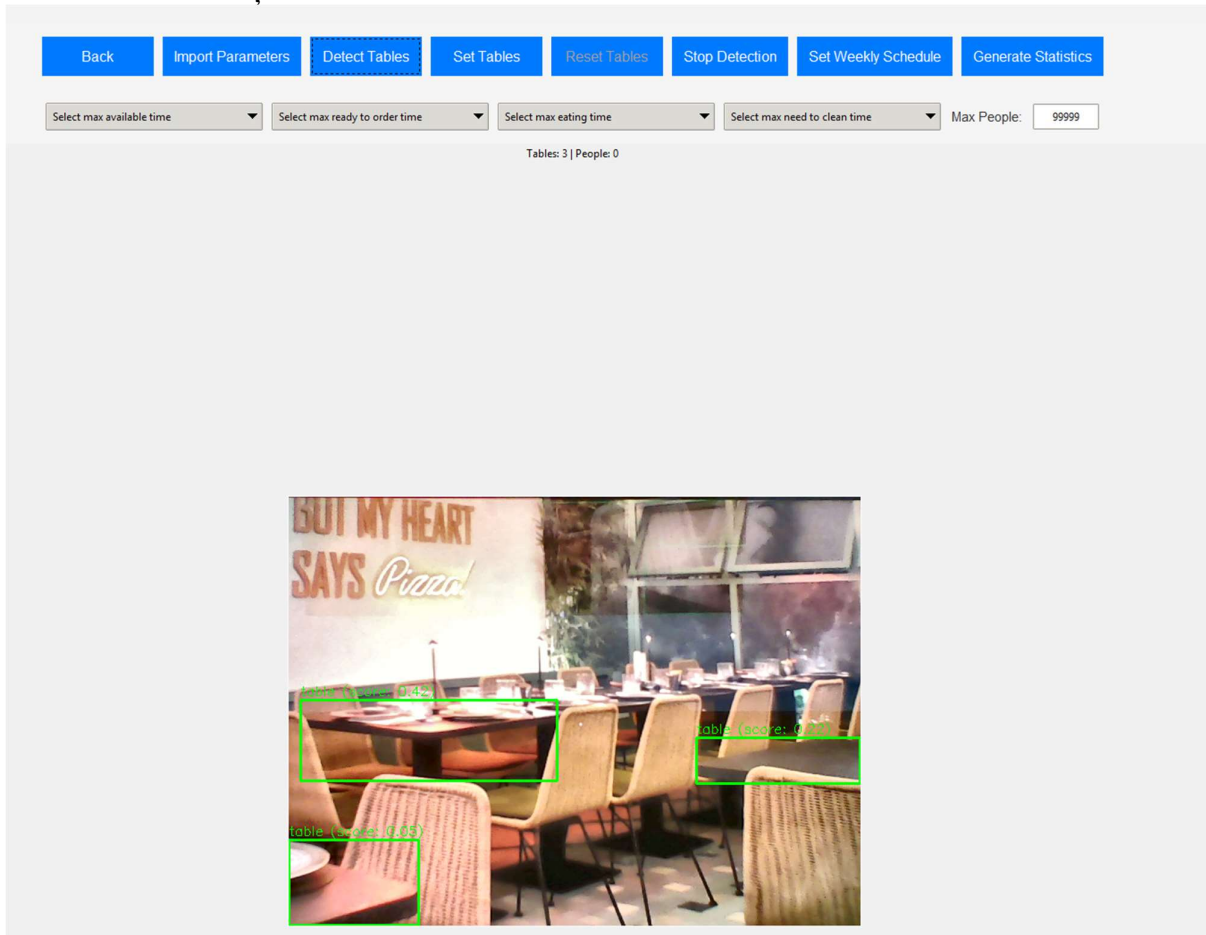


Figura 5.2 – Interfața User GUI

Versiunea destinată utilizatorului final (`user_gui.py`), după cum este ilustrat în Figura 5.2, oferă o interfață simplificată, care încarcă automat fișierul de configurare creat anterior în Admin GUI și inițiază sesiunea de monitorizare. Utilizatorul nu poate modifica parametrii detecției, având rolul de a urmări în timp real rezultatele și de a primi notificări vizuale sau alerte sonore.

5.5.8 Concluzii

Interfața grafică reprezintă un element cheie pentru ușurința utilizării și eficiența sistemului în mediul real. Prin separarea clară a interfeței de administrare față de cea destinată utilizatorului final, aplicația asigură un echilibru optim între flexibilitate, securitate și accesibilitate. Astfel, orice utilizator poate beneficia de funcționalitățile complexe ale sistemului fără a necesita cunoștințe tehnice aprofundate.

5.6 Codul sursă al microcontrolerului ESP32-S3-MINI și integrarea acestuia

5.6.1 Rolul microcontrolerului în arhitectura sistemului

Microcontrolerul ESP32-S3-MINI ocupă un rol central în arhitectura sistemului, asigurând comunicarea dintre aplicația software care rulează pe PC și ansamblul fizic de orientare a camerei video. Programul încărcat pe microcontroler are ca funcție interpretarea comenzilor recepționate prin interfața serială și generarea, prin intermediul bibliotecii ESP32Servo, a semnalelor PWM necesare acționării celor două servomotoare de tip MG996R. Acestea controlează orientarea camerei în planurile orizontal (Ox) și vertical (Oy), contribuind la stabilirea unghiului optim de captare a imaginilor.

Este important de subliniat că acest mecanism de control este activ doar în faza de configurare inițială. Obiectivul său principal este identificarea unei poziții optime a camerei, astfel încât aceasta să capteze un număr maxim de mese în câmpul vizual. Odată ce această poziție a fost stabilită de utilizator, prin intermediul interfeței grafice administrative, camera rămâne fixată, iar microcontrolerul nu mai procesează comenzi noi, cu excepția cazului în care utilizatorul inițiază manual o recalibrare.

Orientarea camerei se poate realiza fie manual, printr-un joystick virtual disponibil în modul `move_camera_frame.py` al interfeței grafice, fie automat, printr-un algoritm de scanare sistematică. În modul automat, camera este deplasată pe o traiectorie spiralată, explorând treptat spațiul disponibil. La fiecare poziție, este calculat un „scor al cadrului” (`frame_score`), care reflectă calitatea unghiului în funcție de numărul și poziționarea meselor detectate în imagine.

Formula de calcul utilizată este următoarea:

```
self.frame_score = (total_score / num_tables) * num_tables if num_tables > 0 else 0
```

Aceasta determină scorul pe baza devierii meselor față de centrul imaginii. La finalul procesului de explorare, cadrul cu scorul maxim este ales drept unghi optim, iar camera este repositionată automat.

5.6.2 Descrierea generală a programului implementat

Codul sursă al microcontrolerului a fost dezvoltat utilizând platforma Arduino IDE, cu ajutorul unui set restrâns de biblioteci dedicate controlului serial și al servomotoarelor. Programul principal se află în directorul `hardware/arduino_servo_camera/arduino_servo_camera.ino`. Structura sa este simplă, fiind împărțită în două secțiuni principale:

- `setup()` – funcția în care sunt inițializate comunicația serială la o viteză de 921600 bauds și sunt configurați pinii asociați celor două servomotoare, prin atașarea acestora la obiectele Servo, ceea ce permite generarea semnalelor PWM necesare poziționării inițiale;
- `loop()` – funcția principală de execuție continuă, în cadrul căreia sunt citite comenzile primite prin interfața serială, interpretate sub forma unghiurilor pentru axele Ox și Oy, și aplicate gradual către cele două servomotoare, asigurând o mișcare lină și controlată a camerei motorizate.